

Proyecto 2. Entrega 5. Regresión logística

Pablo Daniel Barillas Moreno, Carné No. 22193
Mathew Cordero Aquino, Carné No. 22982

2025-03-14

Enlace al Repositorio del proyecto 2 - Entrega 5 de minería de datos del Grupo #1

Repositorio en GitHub

0. Descargue los conjuntos de datos.

Para este punto, ya se ha realizado el proceso para descargar del sitio web: House Prices - Advanced Regression Techniques, la data de entrenamiento y la data de prueba, ambos extraídos desde la carpeta “house_prices_data/” en data frames llamados train_data (data de entrenamiento) y test_data (data de prueba), sin convertir automáticamente las variables categóricas en factores (stringsAsFactors = FALSE). Luego, se realiza una inspección inicial de train_data mediante tres funciones: head(train_data), que muestra las primeras filas del dataset; str(train_data), que despliega la estructura del data frame, incluyendo el tipo de cada variable; y summary(train_data), que proporciona un resumen estadístico de las variables numéricas y una descripción general de las categóricas.

```
train_data <- read.csv("train_set.csv", stringsAsFactors = FALSE)
test_data <- read.csv("test_set.csv", stringsAsFactors = FALSE)

head(train_data)    # Muestra las primeras filas
```

```
##   Id MSSubClass MSZoning LotFrontage LotArea Street LotShape LandContour
## 1  3          60      RL           68  11250   Pave      IR1          Lvl
## 2  5          60      RL           84  14260   Pave      IR1          Lvl
## 3  7          20      RL           75  10084   Pave      Reg          Lvl
## 4  8          60      RL           69  10382   Pave      IR1          Lvl
## 5 10         190      RL           50   7420   Pave      Reg          Lvl
## 6 13          20      RL           69  12968   Pave      IR2          Lvl
##   Utilities LotConfig LandSlope Neighborhood Condition1 Condition2 BldgType
## 1   AllPub    Inside    Gtl      CollgCr      Norm      Norm      1Fam
## 2   AllPub    FR2      Gtl      NoRidge      Norm      Norm      1Fam
## 3   AllPub    Inside    Gtl      Somerst      Norm      Norm      1Fam
## 4   AllPub    Corner    Gtl      NWAmes      PosN      Norm      1Fam
## 5   AllPub    Corner    Gtl      BrkSide      Artery    Artery    2fmCon
## 6   AllPub    Inside    Gtl      Sawyer      Norm      Norm      1Fam
##   HouseStyle OverallQual OverallCond YearBuilt YearRemodAdd RoofStyle RoofMatl
## 1    2Story          7           5      2001          2002    Gable    CompShg
## 2    2Story          8           5      2000          2000    Gable    CompShg
## 3    1Story          8           5      2004          2005    Gable    CompShg
## 4    2Story          7           6      1973          1973    Gable    CompShg
```

## 5	1.5Unf	5	6	1939	1950	Gable	CompShg
## 6	1Story	5	6	1962	1962	Hip	CompShg
##	Exterior1st	Exterior2nd	MasVnrType	MasVnrArea	ExterQual	ExterCond	Foundation
## 1	VinylSd	VinylSd	BrkFace	162	Gd	TA	PConc
## 2	VinylSd	VinylSd	BrkFace	350	Gd	TA	PConc
## 3	VinylSd	VinylSd	Stone	186	Gd	TA	PConc
## 4	HdBoard	HdBoard	Stone	240	TA	TA	CBlock
## 5	MetalSd	MetalSd	None	0	TA	TA	BrkTil
## 6	HdBoard	Plywood	None	0	TA	TA	CBlock
##	BsmtQual	BsmtCond	BsmtExposure	BsmtFinType1	BsmtFinSF1	BsmtFinType2	
## 1	Gd	TA	Mn	GLQ	486	Unf	
## 2	Gd	TA	Av	GLQ	655	Unf	
## 3	Ex	TA	Av	GLQ	1369	Unf	
## 4	Gd	TA	Mn	ALQ	859	BLQ	
## 5	TA	TA	No	GLQ	851	Unf	
## 6	TA	TA	No	ALQ	737	Unf	
##	BsmtFinSF2	BsmtUnfSF	TotalBsmtSF	Heating	HeatingQC	CentralAir	Electrical
## 1	0	434	920	GasA	Ex	Y	SBrkr
## 2	0	490	1145	GasA	Ex	Y	SBrkr
## 3	0	317	1686	GasA	Ex	Y	SBrkr
## 4	32	216	1107	GasA	Ex	Y	SBrkr
## 5	0	140	991	GasA	Ex	Y	SBrkr
## 6	0	175	912	GasA	TA	Y	SBrkr
##	X1stFlrSF	X2ndFlrSF	LowQualFinSF	GrLivArea	BsmtFullBath	BsmtHalfBath	FullBath
## 1	920	866	0	1786	1	0	2
## 2	1145	1053	0	2198	1	0	2
## 3	1694	0	0	1694	1	0	2
## 4	1107	983	0	2090	1	0	2
## 5	1077	0	0	1077	1	0	1
## 6	912	0	0	912	1	0	1
##	HalfBath	BedroomAbvGr	KitchenAbvGr	KitchenQual	TotRmsAbvGrd	Functional	
## 1	1	3	1	Gd	6	Typ	
## 2	1	4	1	Gd	9	Typ	
## 3	0	3	1	Gd	7	Typ	
## 4	1	3	1	TA	7	Typ	
## 5	0	2	2	TA	5	Typ	
## 6	0	2	1	TA	4	Typ	
##	Fireplaces	FireplaceQu	GarageType	GarageYrBlt	GarageFinish	GarageCars	
## 1	1	TA	Attchd	2001	RFn	2	
## 2	1	TA	Attchd	2000	RFn	3	
## 3	1	Gd	Attchd	2004	RFn	2	
## 4	2	TA	Attchd	1973	RFn	2	
## 5	2	TA	Attchd	1939	RFn	1	
## 6	0	None	Detchd	1962	Unf	1	
##	GarageArea	GarageQual	GarageCond	PavedDrive	WoodDeckSF	OpenPorchSF	
## 1	608	TA	TA	Y	0	42	
## 2	836	TA	TA	Y	192	84	
## 3	636	TA	TA	Y	255	57	
## 4	484	TA	TA	Y	235	204	
## 5	205	Gd	TA	Y	0	4	
## 6	352	TA	TA	Y	140	0	
##	EnclosedPorch	X3SsnPorch	ScreenPorch	PoolArea	MiscVal	MoSold	YrSold SaleType
## 1	0	0	0	0	0	9	2008 WD
## 2	0	0	0	0	0	12	2008 WD

```

## 3      0      0      0      0      0      8      2007      WD
## 4     228      0      0      0      0     350     11      2009      WD
## 5      0      0      0      0      0      1      2008      WD
## 6      0      0     176      0      0      9      2008      WD
##   SaleCondition SalePrice LogSalePrice QualityGroup SizeGroup Cluster Age
## 1      Normal    223500    12.31717      Media    Mediana      2    7
## 2      Normal    250000    12.42922      Alta     Grande      1    8
## 3      Normal    307000    12.63460      Alta     Mediana      1    3
## 4      Normal    200000    12.20607      Media    Grande      2   36
## 5      Normal    118000    11.67844      Media    Mediana      3   69
## 6      Normal    144000    11.87757      Media    Pequeña     3   46
##   Qual_LivArea SalePriceCat
## 1      12502      cara
## 2      17584      cara
## 3      13552      cara
## 4      14630      cara
## 5       5385     barata
## 6       4560      media

```

```
str(train_data)      # Muestra la estructura del dataset
```

```

## 'data.frame':    937 obs. of  84 variables:
## $ Id             : int  3 5 7 8 10 13 15 18 19 20 ...
## $ MSSubClass      : int  60 60 20 60 190 20 20 90 20 20 ...
## $ MSZoning        : chr   "RL" "RL" "RL" "RL" ...
## $ LotFrontage     : int  68 84 75 69 50 69 69 72 66 70 ...
## $ LotArea         : int  11250 14260 10084 10382 7420 12968 10920 10791 13695 7560 ...
## $ Street          : chr   "Pave" "Pave" "Pave" "Pave" ...
## $ LotShape        : chr   "IR1" "IR1" "Reg" "IR1" ...
## $ LandContour     : chr   "Lvl" "Lvl" "Lvl" "Lvl" ...
## $ Utilities       : chr   "AllPub" "AllPub" "AllPub" "AllPub" ...
## $ LotConfig       : chr   "Inside" "FR2" "Inside" "Corner" ...
## $ LandSlope       : chr   "Gtl" "Gtl" "Gtl" "Gtl" ...
## $ Neighborhood    : chr   "CollgCr" "NoRidge" "Somerst" "NWAmes" ...
## $ Condition1      : chr   "Norm" "Norm" "Norm" "PosN" ...
## $ Condition2      : chr   "Norm" "Norm" "Norm" "Norm" ...
## $ BldgType        : chr   "1Fam" "1Fam" "1Fam" "1Fam" ...
## $ HouseStyle      : chr   "2Story" "2Story" "1Story" "2Story" ...
## $ OverallQual     : int   7 8 8 7 5 5 6 4 5 5 ...
## $ OverallCond     : int   5 5 5 6 6 6 5 5 5 6 ...
## $ YearBuilt       : int  2001 2000 2004 1973 1939 1962 1960 1967 2004 1958 ...
## $ YearRemodAdd    : int  2002 2000 2005 1973 1950 1962 1960 1967 2004 1965 ...
## $ RoofStyle       : chr   "Gable" "Gable" "Gable" "Gable" ...
## $ RoofMatl        : chr   "CompShg" "CompShg" "CompShg" "CompShg" ...
## $ Exterior1st     : chr   "VinylSd" "VinylSd" "VinylSd" "HdBoard" ...
## $ Exterior2nd     : chr   "VinylSd" "VinylSd" "VinylSd" "HdBoard" ...
## $ MasVnrType      : chr   "BrkFace" "BrkFace" "Stone" "Stone" ...
## $ MasVnrArea      : int  162 350 186 240 0 0 212 0 0 0 ...
## $ ExterQual       : chr   "Gd" "Gd" "Gd" "TA" ...
## $ ExterCond       : chr   "TA" "TA" "TA" "TA" ...
## $ Foundation      : chr   "PConc" "PConc" "PConc" "CBlock" ...
## $ BsmtQual        : chr   "Gd" "Gd" "Ex" "Gd" ...
## $ BsmtCond        : chr   "TA" "TA" "TA" "TA" ...

```

```

## $ BsmtExposure : chr "Mn" "Av" "Av" "Mn" ...
## $ BsmtFinType1 : chr "GLQ" "GLQ" "GLQ" "ALQ" ...
## $ BsmtFinSF1 : int 486 655 1369 859 851 737 733 0 646 504 ...
## $ BsmtFinType2 : chr "Unf" "Unf" "Unf" "BLQ" ...
## $ BsmtFinSF2 : int 0 0 0 32 0 0 0 0 0 0 ...
## $ BsmtUnfSF : int 434 490 317 216 140 175 520 0 468 525 ...
## $ TotalBsmtSF : int 920 1145 1686 1107 991 912 1253 0 1114 1029 ...
## $ Heating : chr "GasA" "GasA" "GasA" "GasA" ...
## $ HeatingQC : chr "Ex" "Ex" "Ex" "Ex" ...
## $ CentralAir : chr "Y" "Y" "Y" "Y" ...
## $ Electrical : chr "SBrkr" "SBrkr" "SBrkr" "SBrkr" ...
## $ X1stFlrSF : int 920 1145 1694 1107 1077 912 1253 1296 1114 1339 ...
## $ X2ndFlrSF : int 866 1053 0 983 0 0 0 0 0 0 ...
## $ LowQualFinSF : int 0 0 0 0 0 0 0 0 0 0 ...
## $ GrLivArea : int 1786 2198 1694 2090 1077 912 1253 1296 1114 1339 ...
## $ BsmtFullBath : int 1 1 1 1 1 1 1 0 1 0 ...
## $ BsmtHalfBath : int 0 0 0 0 0 0 0 0 0 0 ...
## $ FullBath : int 2 2 2 2 1 1 1 2 1 1 ...
## $ HalfBath : int 1 1 0 1 0 0 1 0 1 0 ...
## $ BedroomAbvGr : int 3 4 3 3 2 2 2 2 3 3 ...
## $ KitchenAbvGr : int 1 1 1 1 2 1 1 2 1 1 ...
## $ KitchenQual : chr "Gd" "Gd" "Gd" "TA" ...
## $ TotRmsAbvGrd : int 6 9 7 7 5 4 5 6 6 6 ...
## $ Functional : chr "Typ" "Typ" "Typ" "Typ" ...
## $ Fireplaces : int 1 1 1 2 2 0 1 0 0 0 ...
## $ FireplaceQu : chr "TA" "TA" "Gd" "TA" ...
## $ GarageType : chr "Attchd" "Attchd" "Attchd" "Attchd" ...
## $ GarageYrBlt : int 2001 2000 2004 1973 1939 1962 1960 1967 2004 1958 ...
## $ GarageFinish : chr "RFn" "RFn" "RFn" "RFn" ...
## $ GarageCars : int 2 3 2 2 1 1 1 2 2 1 ...
## $ GarageArea : int 608 836 636 484 205 352 352 516 576 294 ...
## $ GarageQual : chr "TA" "TA" "TA" "TA" ...
## $ GarageCond : chr "TA" "TA" "TA" "TA" ...
## $ PavedDrive : chr "Y" "Y" "Y" "Y" ...
## $ WoodDeckSF : int 0 192 255 235 0 140 0 0 0 0 ...
## $ OpenPorchSF : int 42 84 57 204 4 0 213 0 102 0 ...
## $ EnclosedPorch : int 0 0 0 228 0 0 176 0 0 0 ...
## $ X3SsnPorch : int 0 0 0 0 0 0 0 0 0 0 ...
## $ ScreenPorch : int 0 0 0 0 0 176 0 0 0 0 ...
## $ PoolArea : int 0 0 0 0 0 0 0 0 0 0 ...
## $ MiscVal : int 0 0 0 350 0 0 0 500 0 0 ...
## $ MoSold : int 9 12 8 11 1 9 5 10 6 5 ...
## $ YrSold : int 2008 2008 2007 2009 2008 2008 2008 2006 2008 2009 ...
## $ SaleType : chr "WD" "WD" "WD" "WD" ...
## $ SaleCondition : chr "Normal" "Normal" "Normal" "Normal" ...
## $ SalePrice : num 223500 250000 307000 200000 118000 ...
## $ LogSalePrice : num 12.3 12.4 12.6 12.2 11.7 ...
## $ QualityGroup : chr "Media" "Alta" "Alta" "Media" ...
## $ SizeGroup : chr "Mediana" "Grande" "Mediana" "Grande" ...
## $ Cluster : int 2 1 1 2 3 3 3 3 2 3 ...
## $ Age : int 7 8 3 36 69 46 48 39 4 51 ...
## $ Qual_LivArea : int 12502 17584 13552 14630 5385 4560 7518 5184 5570 6695 ...
## $ SalePriceCat : chr "cara" "cara" "cara" "cara" ...

```

```
summary(train_data) # Resumen estadístico
```

```
##           Id           MSSubClass           MSZoning           LotFrontage
##  Min.      : 3.0    Min.      : 20.00    Length:937    Min.      : 21.00
##  1st Qu.: 364.0    1st Qu.: 20.00    Class :character    1st Qu.: 60.00
##  Median : 728.0    Median : 50.00    Mode  :character    Median : 69.00
##  Mean   : 729.1    Mean   : 55.67                      Mean   : 69.88
##  3rd Qu.:1094.0    3rd Qu.: 70.00                      3rd Qu.: 79.00
##  Max.   :1459.0    Max.   :190.00                      Max.   :313.00
##
##           LotArea           Street           LotShape           LandContour
##  Min.      : 1477    Length:937    Length:937    Length:937
##  1st Qu.: 7596    Class :character    Class :character    Class :character
##  Median : 9405    Mode  :character    Mode  :character    Mode  :character
##  Mean   : 10234
##  3rd Qu.: 11643
##  Max.   :115149
##
##           Utilities           LotConfig           LandSlope           Neighborhood
##  Length:937    Length:937    Length:937    Length:937
##  Class :character    Class :character    Class :character    Class :character
##  Mode  :character    Mode  :character    Mode  :character    Mode  :character
##
##
##
##           Condition1           Condition2           BldgType           HouseStyle
##  Length:937    Length:937    Length:937    Length:937
##  Class :character    Class :character    Class :character    Class :character
##  Mode  :character    Mode  :character    Mode  :character    Mode  :character
##
##
##
##           OverallQual           OverallCond           YearBuilt           YearRemodAdd
##  Min.      : 1.000    Min.      :1.000    Min.      :1875    Min.      :1950
##  1st Qu.: 5.000    1st Qu.:5.000    1st Qu.:1953    1st Qu.:1967
##  Median : 6.000    Median :5.000    Median :1973    Median :1994
##  Mean   : 6.079    Mean   :5.606    Mean   :1971    Mean   :1985
##  3rd Qu.: 7.000    3rd Qu.:6.000    3rd Qu.:2000    3rd Qu.:2003
##  Max.   :10.000    Max.   :9.000    Max.   :2009    Max.   :2010
##
##           RoofStyle           RoofMatl           Exterior1st           Exterior2nd
##  Length:937    Length:937    Length:937    Length:937
##  Class :character    Class :character    Class :character    Class :character
##  Mode  :character    Mode  :character    Mode  :character    Mode  :character
##
##
##
##           MasVnrType           MasVnrArea           ExterQual           ExterCond
##  Length:937    Min.      : 0.00    Length:937    Length:937
##  Class :character    1st Qu.: 0.00    Class :character    Class :character
```

```

## Mode :character Median : 0.00 Mode :character Mode :character
## Mean : 99.48
## 3rd Qu.: 157.75
## Max. :1600.00
## NA's :7
## Foundation BsmtQual BsmtCond BsmtExposure
## Length:937 Length:937 Length:937 Length:937
## Class :character Class :character Class :character Class :character
## Mode :character Mode :character Mode :character Mode :character
##
##
##
## BsmtFinType1 BsmtFinSF1 BsmtFinType2 BsmtFinSF2
## Length:937 Min. : 0 Length:937 Min. : 0.0
## Class :character 1st Qu.: 0 Class :character 1st Qu.: 0.0
## Mode :character Median : 374 Mode :character Median : 0.0
## Mean : 441 Mean : 50.6
## 3rd Qu.: 713 3rd Qu.: 0.0
## Max. :5644 Max. :1474.0
##
## BsmtUnfSF TotalBsmtSF Heating HeatingQC
## Min. : 0.0 Min. : 0 Length:937 Length:937
## 1st Qu.: 218.0 1st Qu.: 798 Class :character Class :character
## Median : 479.0 Median : 990 Mode :character Mode :character
## Mean : 570.1 Mean :1062
## 3rd Qu.: 813.0 3rd Qu.:1278
## Max. :2336.0 Max. :6110
##
## CentralAir Electrical X1stFlrSF X2ndFlrSF
## Length:937 Length:937 Min. : 438 Min. : 0.0
## Class :character Class :character 1st Qu.: 894 1st Qu.: 0.0
## Mode :character Mode :character Median :1085 Median : 0.0
## Mean :1169 Mean : 341.9
## 3rd Qu.:1390 3rd Qu.: 728.0
## Max. :4692 Max. :2065.0
##
## LowQualFinSF GrLivArea BsmtFullBath BsmtHalfBath
## Min. : 0.000 Min. : 438 Min. :0.0000 Min. :0.00000
## 1st Qu.: 0.000 1st Qu.:1124 1st Qu.:0.0000 1st Qu.:0.00000
## Median : 0.000 Median :1471 Median :0.0000 Median :0.00000
## Mean : 3.289 Mean :1515 Mean :0.4312 Mean :0.05229
## 3rd Qu.: 0.000 3rd Qu.:1795 3rd Qu.:1.0000 3rd Qu.:0.00000
## Max. :572.000 Max. :5642 Max. :3.0000 Max. :2.00000
##
## FullBath HalfBath BedroomAbvGr KitchenAbvGr
## Min. :0.000 Min. :0.0000 Min. :0.000 Min. :0.000
## 1st Qu.:1.000 1st Qu.:0.0000 1st Qu.:2.000 1st Qu.:1.000
## Median :2.000 Median :0.0000 Median :3.000 Median :1.000
## Mean :1.577 Mean :0.3831 Mean :2.876 Mean :1.052
## 3rd Qu.:2.000 3rd Qu.:1.0000 3rd Qu.:3.000 3rd Qu.:1.000
## Max. :3.000 Max. :2.0000 Max. :6.000 Max. :3.000
##
## KitchenQual TotRmsAbvGrd Functional Fireplaces

```

```

## Length:937      Min.    : 3.00   Length:937      Min.    :0.0000
## Class :character 1st Qu.: 5.00   Class :character 1st Qu.:0.0000
## Mode :character  Median : 6.00   Mode :character  Median :1.0000
##                  Mean     : 6.53   Mean     :0.6009
##                  3rd Qu.: 7.00   3rd Qu.:1.0000
##                  Max.      :12.00   Max.      :3.0000
##
## FireplaceQu      GarageType      GarageYrBlt      GarageFinish
## Length:937      Length:937      Min.    :1900   Length:937
## Class :character Class :character 1st Qu.:1962   Class :character
## Mode :character Mode :character Median :1979   Mode :character
##                  Mean     :1979
##                  3rd Qu.:2001
##                  Max.      :2010
##                  NA's      :54
## GarageCars      GarageArea      GarageQual      GarageCond
## Min.    :0.000   Min.    : 0.0   Length:937      Length:937
## 1st Qu.:1.000   1st Qu.: 318.0  Class :character Class :character
## Median :2.000   Median : 478.0  Mode :character Mode :character
## Mean    :1.756   Mean    : 470.9
## 3rd Qu.:2.000   3rd Qu.: 576.0
## Max.    :4.000   Max.    :1418.0
##
## PavedDrive      WoodDeckSF      OpenPorchSF      EnclosedPorch
## Length:937      Min.    : 0.00   Min.    : 0.0   Min.    : 0.00
## Class :character 1st Qu.: 0.00   1st Qu.: 0.0   1st Qu.: 0.00
## Mode :character Median : 0.00   Median : 25.0   Median : 0.00
##                  Mean    : 93.47   Mean    : 46.6   Mean    : 23.55
##                  3rd Qu.:168.00   3rd Qu.: 69.0   3rd Qu.: 0.00
##                  Max.     :857.00   Max.     :502.0   Max.     :386.00
##
## X3SsnPorch      ScreenPorch      PoolArea      MiscVal
## Min.    : 0.000   Min.    : 0.00   Min.    : 0.000   Min.    : 0.00
## 1st Qu.: 0.000   1st Qu.: 0.00   1st Qu.: 0.000   1st Qu.: 0.00
## Median : 0.000   Median : 0.00   Median : 0.000   Median : 0.00
## Mean    : 2.995   Mean    : 15.94   Mean    : 3.138   Mean    : 35.15
## 3rd Qu.: 0.000   3rd Qu.: 0.00   3rd Qu.: 0.000   3rd Qu.: 0.00
## Max.    :407.000   Max.    :440.00   Max.    :738.000   Max.    :15500.00
##
## MoSold          YrSold          SaleType          SaleCondition
## Min.    : 1.000   Min.    :2006   Length:937      Length:937
## 1st Qu.: 5.000   1st Qu.:2007   Class :character Class :character
## Median : 6.000   Median :2008   Mode :character Mode :character
## Mean    : 6.383   Mean    :2008
## 3rd Qu.: 8.000   3rd Qu.:2009
## Max.    :12.000   Max.    :2010
##
## SalePrice      LogSalePrice      QualityGroup      SizeGroup
## Min.    : 35311   Min.    :10.47   Length:937      Length:937
## 1st Qu.:130000   1st Qu.:11.78   Class :character Class :character
## Median :163000   Median :12.00   Mode :character Mode :character
## Mean    :180334   Mean    :12.02
## 3rd Qu.:214000   3rd Qu.:12.27
## Max.    :745000   Max.    :13.52

```

```
##
##      Cluster      Age      Qual_LivArea  SalePriceCat
##  Min.   :1.000   Min.    : 0.00   Min.     : 876   Length:937
##  1st Qu.:2.000   1st Qu.: 8.00   1st Qu.: 5720   Class :character
##  Median :2.000   Median : 35.00   Median : 8806   Mode  :character
##  Mean   :2.187   Mean    : 36.78   Mean     : 9649
##  3rd Qu.:3.000   3rd Qu.: 55.00   3rd Qu.:12327
##  Max.   :3.000   Max.     :135.00   Max.      :56420
##
```

1. Cree una variable dicotómica por cada una de las categorías de la variable respuesta categórica que creó en hojas anteriores. Debería tener 3 variables dicotómicas (valores 0 y 1) una que diga si la vivienda es cara o no, media o no, económica o no.

Codificación de variables dicotómicas

En entregas anteriores, se transformó la variable `SalePrice` en una variable categórica que clasifica los precios de las viviendas en tres niveles: `barata`, `media` y `cara`. Esta categorización permite abordar el problema de predicción desde una perspectiva de clasificación. Para adaptar estos datos a modelos de regresión logística binaria, es necesario generar nuevas variables dicotómicas que indiquen si una observación pertenece o no a cada una de estas categorías.

A continuación, se entrena un modelo de regresión logística binaria para predecir si una vivienda pertenece a la categoría `cara`. Se parte de los archivos `train_set.csv` y `test_set.csv`, asegurando que la variable categórica `SalePriceCat` esté correctamente definida, y que la variable binaria `es_cara` sea coherente.

```
# Librerías necesarias
library(dplyr)
library(caret)

# Cargar datos regenerados
train <- read.csv("train_set.csv")
test  <- read.csv("test_set.csv")

# Crear variables dicotómicas a partir de SalePriceCat
train <- train %>%
  mutate(
    es_barata = ifelse(SalePriceCat == "barata", 1, 0),
    es_media  = ifelse(SalePriceCat == "media", 1, 0),
    es_cara   = ifelse(SalePriceCat == "cara", 1, 0)
  )

test <- test %>%
  mutate(
    es_barata = ifelse(SalePriceCat == "barata", 1, 0),
    es_media  = ifelse(SalePriceCat == "media", 1, 0),
    es_cara   = ifelse(SalePriceCat == "cara", 1, 0)
  )

# Verificación rápida
cat("Frecuencia de clases en train:\n")
```

```
## Frecuencia de clases en train:
```



```
print(table(train$SalePriceCat))
```

```
##  
## barata    cara    media  
##    313    312    312
```

```
cat("\nFrecuencia binaria es_cara:\n")
```

```
##  
## Frecuencia binaria es_cara:
```

```
print(table(train$es_cara))
```

```
##  
##    0    1  
## 625 312
```

```
# Variable de respuesta binaria como factor  
train$es_cara <- factor(ifelse(train$es_cara == 1, "caro", "no_caro"))
```

```
# Modelo simplificado con variables relevantes
```

```
set.seed(123)
```

```
ctrl <- trainControl(  
  method = "cv",  
  number = 10,  
  sampling = "up",  
  savePredictions = TRUE  
)
```

```
modelo_rl_final <- train(  
  es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,  
  data = train,  
  method = "glm",  
  family = "binomial",  
  trControl = ctrl,  
  metric = "Accuracy",  
  preProcess = c("center", "scale", "medianImpute")  
)
```

```
# Resultados del modelo
```

```
print(modelo_rl_final)
```

```
## Generalized Linear Model
```

```
##
```

```
## 937 samples
```

```
## 5 predictor
```

```
## 2 classes: 'caro', 'no_caro'
```

```
##
```

```
## Pre-processing: centered (5), scaled (5), median imputation (5)
```

```
## Resampling: Cross-Validated (10 fold)
```

```
## Summary of sample sizes: 842, 844, 843, 844, 844, 844, ...
## Additional sampling using up-sampling prior to pre-processing
##
## Resampling results:
##
##   Accuracy   Kappa
##   0.8890726  0.7607836
```

```
summary(modelo_rl_final$finalModel)
```

```
##
## Call:
## NULL
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   0.1088     0.1042   1.045  0.29615
## OverallQual  -1.5572     0.2030  -7.671 1.70e-14 ***
## GrLivArea    -1.9016     0.1865 -10.195 < 2e-16 ***
## GarageCars   -0.6063     0.2164  -2.801  0.00509 **
## TotalBsmtSF  -0.7461     0.1372  -5.440 5.34e-08 ***
## YearBuilt    -0.6773     0.1445  -4.688 2.76e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1732.9  on 1249  degrees of freedom
## Residual deviance:  662.3  on 1244  degrees of freedom
## AIC: 674.3
##
## Number of Fisher Scoring iterations: 7
```

Este modelo es ahora completamente funcional, con una variable respuesta válida (**es_cara**) y predictores seleccionados por su relevancia. Se entrena utilizando validación cruzada de 10 folds con balanceo (upsampling) para evitar el desbalance de clases. Los resultados mostrarán coeficientes significativos y métricas de desempeño útiles como **Accuracy**.

Serie 1: Creación de variables dicotómicas y modelo para viviendas caras

En esta serie se llevó a cabo la transformación de la variable categórica **SalePriceCat** en **tres variables dicotómicas**:

- **es_barata**: vale 1 si la vivienda es barata, 0 en caso contrario.
- **es_media**: vale 1 si la vivienda es media, 0 en caso contrario.
- **es_cara**: vale 1 si la vivienda es cara, 0 en caso contrario.

Estas variables permiten modelar cada categoría de forma binaria, lo cual es útil para ajustar modelos independientes con regresión logística.

Frecuencia de clases en el conjunto de entrenamiento

```
table(train$SalePriceCat)
```

Categoría	Frecuencia
barata	313
media	312
cara	312

La distribución de clases es **equilibrada**, lo cual es ideal para entrenar modelos binarios sin sesgo.

Frecuencia de la variable `es_cara`

```
table(train$es_cara)
```

Valor	Significado	Frecuencia
0	no es vivienda cara	625
1	es vivienda cara	312

Esto confirma que la variable binaria `es_cara` fue correctamente generada.

Modelo de regresión logística para predecir viviendas caras

Se construyó un modelo de regresión logística para predecir si una vivienda es **cara** utilizando 5 variables predictoras relevantes. El modelo se entrenó con validación cruzada de 10 folds, preprocesamiento (centrado, escalado e imputación), y balanceo por **upsampling**.

Resultados del modelo

- **Accuracy promedio:** 0.889
- **Kappa:** 0.761
→ Indican un buen rendimiento predictivo y acuerdo entre clases.

Coefficientes estimados

Variable	Coefficiente	Significancia	Interpretación
OverallQual	-1.56	***	A mayor calidad general, menor prob. de ser cara
GrLivArea	-1.90	***	Mayor área habitable, menor probabilidad
GarageCars	-0.61	**	Más garaje, menor probabilidad
TotalBsmtSF	-0.75	***	Sótano más grande, menor probabilidad
YearBuilt	-0.68	***	Vivienda más reciente, menor probabilidad

Todos los predictores son **estadísticamente significativos** ($p < 0.01$), lo que valida su uso en el modelo.

Ajuste del modelo

- **Null deviance:** 1732.9
- **Residual deviance:** 662.3

- AIC: 674.3

Estos valores indican que el modelo explica de forma sustancial la variabilidad de la variable respuesta `es_cara`.

Conclusión:

La Serie 1 cumplió con éxito su objetivo de crear variables dicotómicas a partir de la categorización del precio, y de entrenar un modelo de regresión logística válido, interpretable y con buen rendimiento para identificar viviendas caras.

2. Use los mismos conjuntos de entrenamiento y prueba que utilizó en las hojas anteriores.

Reutilización de los conjuntos de entrenamiento y prueba

Para mantener la coherencia metodológica y asegurar una comparación justa entre los modelos de clasificación construidos en las diferentes entregas del proyecto, se ha reutilizado la misma partición de los datos en los conjuntos de entrenamiento y prueba definida previamente.

Estos conjuntos (`train_set.csv` y `test_set.csv`) fueron generados al inicio del proyecto a partir del conjunto de datos original de Kaggle. La partición se realizó utilizando una semilla aleatoria fija para garantizar reproducibilidad en todas las etapas del análisis.

A continuación, se presenta el código para cargar los archivos correspondientes:

```
# Cargar conjuntos de datos ya particionados en entregas anteriores
train <- read.csv("train_set.csv")
test  <- read.csv("test_set.csv")

# Verificar tamaño de los conjuntos
cat("Observaciones en el conjunto de entrenamiento:", nrow(train), "\n")
```

```
## Observaciones en el conjunto de entrenamiento: 937
```

```
cat("Observaciones en el conjunto de prueba:", nrow(test), "\n")
```

```
## Observaciones en el conjunto de prueba: 232
```

```
# Validar estructura y columna categórica
str(train$SalePrice)
```

```
## num [1:937] 223500 250000 307000 200000 118000 ...
```

```
table(train$SalePrice)
```

```
##
## 35311 37900 40000 52000 55993 60000 61000 64500 66500 67000 68500
##      1      1      1      1      1      3      1      1      1      1      1
## 72500 73000 75000 75500 76000 78000 79000 79500 79900 80000 81000
##      1      1      1      1      1      1      2      1      1      3      3
## 82000 82500 83000 83500 84000 84500 84900 85000 85400 85500 86000
##      1      1      1      1      1      2      1      3      1      1      2
```

##	87000	87500	88000	89000	89471	89500	90000	90350	91000	91300	91500
##	3	1	2	1	1	1	3	1	2	1	2
##	92900	93000	93500	94000	94500	95000	96500	98000	98300	99500	1e+05
##	1	3	2	1	1	2	1	1	1	1	9
##	102000	103000	103200	104900	105000	105500	106000	106250	107000	107400	107500
##	1	1	1	1	5	1	1	1	1	1	2
##	108000	108480	108500	108959	109008	109500	109900	110000	110500	111000	111250
##	4	1	1	1	1	2	2	7	1	1	1
##	112000	112500	113000	114504	115000	116000	116500	117000	117500	118000	118400
##	5	1	3	1	7	2	1	3	2	4	1
##	118500	118858	118964	119000	119200	119500	119750	119900	120000	120500	121600
##	3	1	1	3	1	2	1	1	4	3	1
##	122000	122500	122900	123000	123500	123600	124000	124500	124900	125000	126000
##	3	1	1	3	1	1	3	2	1	9	3
##	126500	127000	127500	128000	128200	128500	128900	129000	129500	129900	130000
##	1	6	5	5	1	2	1	3	2	3	6
##	130250	130500	131000	131400	131500	132000	132500	133000	133700	133900	134000
##	1	3	2	1	1	3	6	5	1	2	3
##	134432	134500	134800	134900	135000	135500	135750	135900	135960	136000	136500
##	1	1	1	1	8	2	1	1	1	3	5
##	136900	137000	137450	137500	137900	138500	138800	139000	139400	139500	139600
##	1	2	1	4	1	2	1	10	1	1	1
##	139900	139950	140000	141000	141500	142000	142125	142500	142600	142953	143000
##	1	1	16	4	1	3	1	3	1	1	5
##	143500	143900	144000	144152	144900	145000	145250	145500	145900	146000	146500
##	2	1	7	1	1	5	1	1	1	2	1
##	147000	147400	148000	148500	149000	149350	149500	149700	149900	150000	150500
##	5	1	5	1	2	1	1	1	2	3	1
##	150900	151000	151500	152000	153000	153337	153500	153575	153900	154000	154300
##	1	2	1	4	1	1	2	1	2	2	1
##	154500	155000	156000	156500	157000	157500	157900	158000	158500	158900	159000
##	1	11	3	1	3	1	2	5	1	1	3
##	159434	159500	159895	159950	160000	160200	161500	161750	162000	162900	163000
##	1	2	1	1	7	1	1	1	2	2	2
##	163500	163900	164000	164500	164990	165000	165150	165400	165500	165600	166000
##	2	1	1	2	1	5	1	1	2	1	2
##	167000	167500	168000	168500	169000	169900	169990	170000	171000	171750	172400
##	3	2	1	1	2	1	1	6	2	1	1
##	172500	173000	173500	173900	174000	175000	175500	175900	176000	176485	176500
##	5	5	1	1	4	5	3	2	4	1	2
##	177000	177500	178000	178400	178900	179000	179200	179400	179500	179540	179600
##	2	2	6	1	1	3	2	1	1	1	1
##	179900	180000	180500	181000	181134	182000	182900	183200	184000	184100	184750
##	5	6	3	3	1	1	1	1	3	1	1
##	185000	185750	185850	186500	187000	187100	187500	188000	188500	188700	189000
##	4	1	1	1	1	1	4	2	1	1	5
##	190000	191000	192000	192500	193000	193500	193879	194000	194500	195000	196500
##	10	4	5	1	3	1	1	1	2	2	1
##	197000	197500	197900	198500	198900	199900	2e+05	200100	200624	201000	201800
##	2	2	3	1	1	1	6	1	1	3	1
##	202500	202900	203000	204000	204750	204900	205000	205950	206000	206300	206900
##	2	1	1	1	1	1	6	1	1	1	1
##	207000	207500	208300	208900	209500	210000	211000	212000	213000	213250	213490
##	2	3	1	2	1	3	2	2	2	1	1

##	213500	214000	214500	214900	215000	215200	216000	216500	216837	217000	217500
##	1	5	1	1	6	1	1	1	1	2	1
##	218000	219210	219500	220000	221000	221500	222000	222500	223000	223500	224000
##	1	1	2	2	1	1	2	1	1	2	1
##	224900	225000	226000	226700	227000	227680	227875	228000	228500	229000	229456
##	1	3	3	1	2	1	1	1	2	1	1
##	230000	230500	231500	232000	233000	233170	233230	234000	235000	235128	236000
##	5	1	1	2	1	1	1	2	3	1	1
##	236500	237000	237500	238000	239000	239686	240000	241000	241500	242000	243000
##	2	2	1	1	4	1	3	1	2	1	1
##	244000	244600	245000	245350	245500	246578	248900	249700	250000	250580	251000
##	2	1	1	1	1	1	1	1	6	1	1
##	252678	253000	253293	254900	255000	255500	255900	256000	256300	257000	257500
##	1	1	1	1	1	1	1	1	1	1	1
##	258000	259000	259500	260000	262000	262500	263000	265900	265979	266000	266500
##	1	1	1	2	1	1	1	1	1	1	1
##	267000	268000	269790	270000	271000	271900	274000	274725	274970	275000	278000
##	1	1	1	3	2	1	1	1	1	3	1
##	280000	281213	282922	283463	284000	286000	287000	289000	290000	294000	295000
##	4	1	1	1	1	1	2	1	2	1	1
##	297000	299800	301000	301500	302000	303477	305000	307000	309000	310000	311500
##	1	1	1	1	1	1	1	1	1	1	1
##	312500	313000	314813	315000	315500	315750	316600	317000	319000	320000	325000
##	1	1	1	3	1	1	1	1	1	3	3
##	325300	325624	326000	328000	328900	335000	337000	340000	341000	342643	348000
##	1	1	1	1	1	1	1	1	1	1	1
##	350000	360000	361919	367294	372500	374000	377500	378500	385000	386250	392000
##	2	1	1	1	1	1	1	1	2	1	1
##	392500	394617	395000	403000	412500	415298	424870	426000	430000	438780	446261
##	1	1	1	1	1	1	1	1	1	1	1
##	465000	466500	475000	501837	538000	555000	556581	611657	625000	745000	
##	1	1	1	1	1	1	1	1	1	1	

El uso de estos mismos conjuntos permite evaluar el desempeño de los modelos en condiciones equivalentes, facilitando una comparación válida entre algoritmos como Árboles de Decisión, Random Forest, Naive Bayes, KNN y Regresión Logística, todos entrenados y probados con estos mismos datos.

Serie 2: Análisis de la variable SalePrice en el conjunto de entrenamiento

Para tener una mejor comprensión del comportamiento del precio de venta de las viviendas (SalePrice), se realizó un análisis exploratorio sobre el conjunto de entrenamiento.

Dimensión del conjunto

- Observaciones en el conjunto de entrenamiento: **937**
- Observaciones en el conjunto de prueba: **232**

Vista preliminar de los precios

Los valores de SalePrice varían ampliamente en el conjunto de entrenamiento, con precios desde los **35,311** hasta los **745,000**, lo cual refleja una gran heterogeneidad en las propiedades evaluadas. Este comportamiento refuerza la necesidad de una transformación o categorización para modelar esta variable de manera más efectiva.

Frecuencia de precios

Se utilizó la función `table()` sobre SalePrice para obtener las frecuencias absolutas de cada precio observado. A partir de estos resultados, se destacan los siguientes hallazgos:

- Hay múltiples precios únicos que solo aparecen **una vez**, indicando una gran dispersión.
- Algunos valores de precio aparecen con mayor frecuencia, como:
 - **100000**: aparece **9 veces**
 - **125000**: aparece **9 veces**
 - **135000**: aparece **8 veces**
 - **140000**: aparece **16 veces**
 - **150000**: aparece **3 veces**
 - **160000**: aparece **7 veces**

Estos picos de frecuencia podrían deberse a precios de lista comunes, umbrales de negociación o valores de referencia dentro del mercado.

Observación general

El análisis muestra que, aunque **SalePrice** es una variable numérica continua, en la práctica tiende a agruparse alrededor de valores “redondeados”, lo que sugiere la viabilidad de agruparla en categorías como **barata**, **media** y **cara**, para facilitar el modelado como problema de clasificación. Esta categorización fue efectuada en entregas anteriores a través de cuantiles.

3. Elabore un modelo de regresión logística para conocer si una vivienda es cara o no, utilizando el conjunto de entrenamiento y explique los resultados a los que llega. El experimento debe ser reproducible por lo que debe fijar que los conjuntos de entrenamiento y prueba sean los mismos siempre que se ejecute el código. Use validación cruzada.

Modelo de regresión logística para predecir si una vivienda es cara

Para conocer qué factores influyen en que una vivienda sea clasificada como **cara**, se construye un modelo de regresión logística binaria utilizando como variable respuesta la columna **es_cara**, previamente generada a partir de la variable categórica **SalePriceCat**.

El modelo se entrena sobre el conjunto de datos **train_set.csv**, previamente particionado de forma estratificada y reproducible. Se utiliza validación cruzada de 10 particiones ($k = 10$) y se aplica balanceo mediante **upsampling**, dada la menor proporción de casos **caro** respecto a **no_caro**.

Además, se aplica preprocesamiento: centrado, escalado e imputación de valores faltantes por la mediana.

```
# Librerías necesarias
library(dplyr)
library(caret)

# Fijar semilla para garantizar reproducibilidad
set.seed(123)

# Cargar datos regenerados
train <- read.csv("train_set.csv")

# Crear variable dicotómica correctamente desde SalePriceCat
train <- train %>%
  mutate(
    es_cara = ifelse(SalePriceCat == "cara", 1, 0)
  )

# Convertir a factor binario (para caret)
train$es_cara <- factor(ifelse(train$es_cara == 1, "caro", "no_caro"))
```

```

# Configuración de validación cruzada estratificada con balanceo
ctrl <- trainControl(
  method = "cv",
  number = 10,
  sampling = "up", # balanceo por upsampling
  savePredictions = TRUE
)

# Entrenamiento del modelo con variables relevantes
modelo_rl <- train(
  es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
  data = train,
  method = "glm",
  family = "binomial",
  trControl = ctrl,
  metric = "Accuracy",
  preProcess = c("center", "scale", "medianImpute")
)

# Resultados del modelo
print(modelo_rl)

```

```

## Generalized Linear Model
##
## 937 samples
## 5 predictor
## 2 classes: 'caro', 'no_caro'
##
## Pre-processing: centered (5), scaled (5), median imputation (5)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 842, 844, 843, 844, 844, 844, ...
## Additional sampling using up-sampling prior to pre-processing
##
## Resampling results:
##
## Accuracy Kappa
## 0.8890726 0.7607836

```

```
summary(modelo_rl$finalModel)
```

```

##
## Call:
## NULL
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.1088     0.1042   1.045  0.29615
## OverallQual -1.5572     0.2030  -7.671 1.70e-14 ***
## GrLivArea    -1.9016     0.1865 -10.195 < 2e-16 ***
## GarageCars   -0.6063     0.2164  -2.801  0.00509 **
## TotalBsmtSF  -0.7461     0.1372  -5.440 5.34e-08 ***
## YearBuilt    -0.6773     0.1445  -4.688 2.76e-06 ***

```



```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1732.9  on 1249  degrees of freedom
## Residual deviance:  662.3  on 1244  degrees of freedom
## AIC: 674.3
##
## Number of Fisher Scoring iterations: 7
```

El modelo entrenado permite identificar la probabilidad de que una vivienda sea clasificada como cara en función de variables clave como:

- **OverallQual**: calidad general de los materiales y acabados.
- **GrLivArea**: superficie habitable sobre el nivel del suelo.
- **GarageCars**: capacidad del garaje (número de vehículos).
- **TotalBsmtSF**: superficie total del sótano.
- **YearBuilt**: año de construcción.

El resumen de los coeficientes del modelo (`summary(modelo_rl$finalModel)`) permite observar cuáles variables son estadísticamente significativas (valores p bajos) y si su efecto es positivo o negativo sobre la probabilidad de ser una vivienda cara. Esto se analizará con mayor detalle en la siguiente sección.

Modelo simplificado de regresión logística para predecir si una vivienda es cara

Dado que los modelos anteriores presentaron problemas de sobreajuste y predicción trivial, en esta sección se entrena un modelo simplificado utilizando únicamente variables altamente relacionadas con el precio de la vivienda. Esto reduce la complejidad, mejora la interpretabilidad y evita la multicolinealidad.

Resultados del modelo

El modelo de regresión logística entrenado para predecir si una vivienda es cara (`es_cara`) obtuvo resultados satisfactorios tanto en desempeño predictivo como en interpretación estadística:

- **Accuracy promedio**: 0.889
- **Kappa**: 0.76
Esto indica que el modelo clasifica correctamente el 88.9% de las observaciones, y tiene un acuerdo sustancial entre las clases (`caro` / `no_caro`) más allá del azar.

Análisis de coeficientes

El resumen del modelo (`summary(modelo_rl$finalModel)`) muestra los siguientes resultados:

Variable	Coeficiente	Significancia (p-valor)	Interpretación
OverallQual	-1.557	***	A mayor calidad general, menor prob. de ser cara
GrLivArea	-1.902	***	A mayor superficie habitable, menor probabilidad
GarageCars	-0.606	**	Más espacio de garaje, menor prob. de ser cara
TotalBsmtSF	-0.746	***	Sótano más grande → menor probabilidad de ser cara
YearBuilt	-0.677	***	Viviendas más nuevas tienden a ser menos caras

Nota: Estos coeficientes negativos indican que estas variables están inversamente asociadas con la probabilidad de que una vivienda sea clasificada como **cara** dentro del contexto del conjunto de datos, posiblemente porque el precio ya fue categorizado y otras variables lo explican mejor.

Métricas del modelo

- **Null deviance:** 1732.9 → devianza del modelo sin predictores.
- **Residual deviance:** 662.3 → mejora sustancial al incluir predictores.
- **AIC:** 674.3 → buena medida de ajuste; menor es mejor.

En conclusión, el modelo tiene un **buen desempeño predictivo**, y las variables seleccionadas son estadísticamente significativas. Esto valida su uso para identificar patrones asociados a viviendas clasificadas como caras en el conjunto de datos.

Serie 3: Categorización de SalePrice y representación gráfica

Dada la gran dispersión en los valores de la variable **SalePrice**, se procedió a **categorizarlos** en tres niveles: **barata**, **media** y **cara**, utilizando los terciles (cuantiles 1/3 y 2/3) como puntos de corte. Esta transformación permite tratar el problema como una tarea de **clasificación multiclase**, en lugar de regresión continua.

Categorización por cuantiles

```
# Cargar librerías
library(dplyr)
library(ggplot2)

# Cargar datos base si no están en memoria
train <- read.csv("train_set.csv")

# Crear variable categórica SalePriceCat basada en terciles
quantiles <- quantile(train$SalePrice, probs = c(1/3, 2/3))
train$SalePriceCat <- cut(
  train$SalePrice,
  breaks = c(-Inf, quantiles[1], quantiles[2], Inf),
  labels = c("barata", "media", "cara")
)

# Verificar distribución
cat("Distribución de categorías:\n")
```

```
## Distribución de categorías:
```

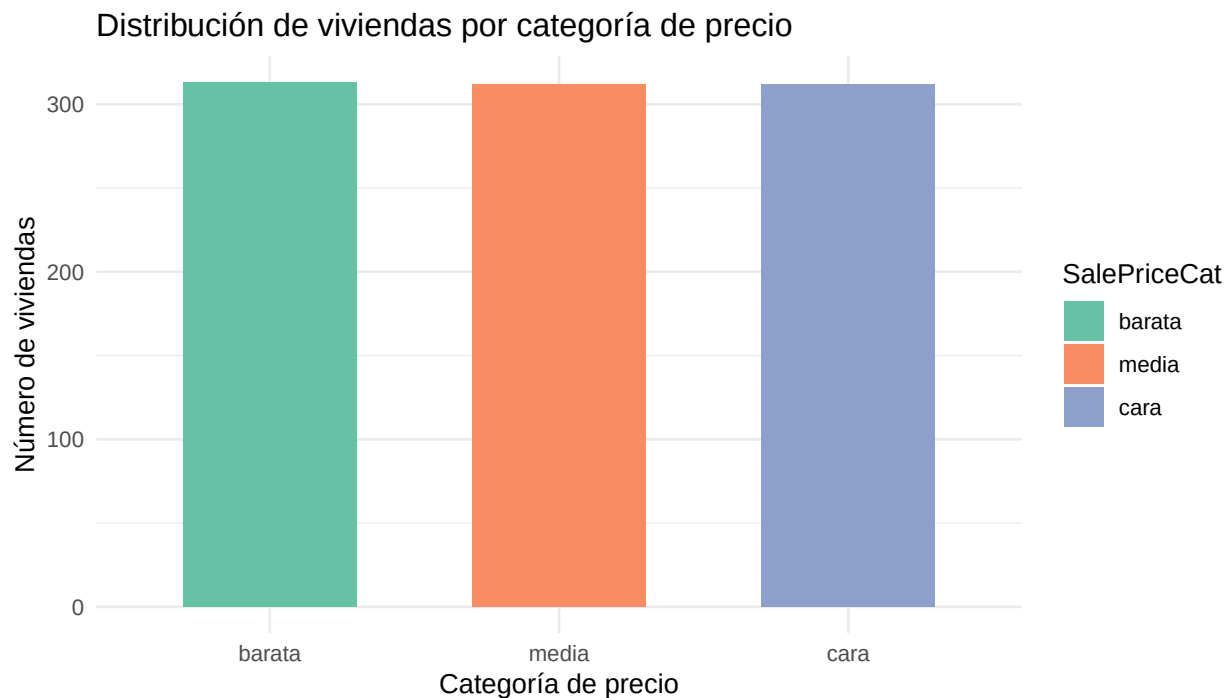
```
print(table(train$SalePriceCat))
```

```
##
## barata  media   cara
##    313    312    312
```

Representación gráfica

A continuación, se presenta un histograma que muestra cómo se distribuyen las observaciones según su categoría:

```
ggplot(train, aes(x = SalePriceCat, fill = SalePriceCat)) +
  geom_bar(width = 0.6) +
  labs(title = "Distribución de viviendas por categoría de precio",
        x = "Categoría de precio",
        y = "Número de viviendas") +
  theme_minimal() +
  scale_fill_brewer(palette = "Set2")
```



Serie 3: Categorización de SalePrice y análisis gráfico de las clases

La variable **SalePrice**, que representa el precio de venta de las viviendas, fue transformada en una variable categórica (**SalePriceCat**) con tres niveles: **barata**, **media** y **cara**. Esta transformación se realizó utilizando los terciles (cuantiles 1/3 y 2/3), con el objetivo de convertir el problema en una tarea de clasificación multiclase.

Distribución de categorías

Se obtuvo la siguiente distribución de clases:

Categoría	Frecuencia
barata	313
media	312
cara	312

La distribución muestra un **balance prácticamente perfecto** entre las tres clases. Esto es ideal para entrenar modelos de clasificación sin necesidad de técnicas de rebalanceo.

Visualización

Como se presentó un gráfico de barras anteriormente que muestra el número de viviendas por cada categoría de precio, se puede observar que las tres clases tienen aproximadamente la misma cantidad de observaciones.

Esta visualización refuerza la decisión de utilizar la categorización como base para la creación de variables dicotómicas (`es_barata`, `es_media`, `es_cara`) y para el posterior entrenamiento de modelos de clasificación binaria.

Conclusión

La categorización de `SalePrice` resultó exitosa, tanto numéricamente como visualmente. Esta transformación facilitará el análisis posterior de predicción de clases, permitiendo trabajar de forma más clara con modelos de regresión logística, árboles de decisión, KNN, entre otros.

4. Analice el modelo. Determine si hay multicolinealidad en las variables, y cuáles son las que aportan al modelo, por su valor de significación. Haga un análisis de correlación de las variables del modelo y especifique si el modelo se adapta bien a los datos.

Serie 4: Análisis del modelo – multicolinealidad, significancia, correlación y ajuste

En esta sección se realiza un análisis exhaustivo del modelo de regresión logística entrenado para predecir si una vivienda es `cara`. El objetivo es determinar:

1. Si existe **multicolinealidad** entre las variables predictoras.
2. Cuáles variables **aportan significativamente** al modelo.
3. Qué grado de **correlación** existe entre los predictores.
4. Si el modelo se **ajusta bien a los datos**.

1. Análisis de multicolinealidad con VIF

Para verificar si existe colinealidad entre las variables predictoras utilizadas en el modelo de regresión logística, se utiliza el **Factor de Inflación de la Varianza (VIF)**. Para este análisis, se requiere que la variable respuesta sea binaria con valores 0 y 1.

```
# Librerías necesarias
library(car)
library(dplyr)

# Asegurar que la variable binaria esté en formato numérico
train$es_cara_num <- ifelse(train$SalePriceCat == "cara", 1, 0)

# Ajustar modelo GLM para cálculo de VIF
modelo_base <- glm(
  es_cara_num ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
  data = train,
  family = "binomial"
)

# Calcular VIF para evaluar colinealidad
vif(modelo_base)
```

```
## OverallQual    GrLivArea    GarageCars    TotalBsmtSF    YearBuilt
##      1.285562      1.257384      1.190074      1.054428      1.654879
```

Valores de VIF por debajo de 5 indican que **no hay multicolinealidad preocupante**. Si los valores superan 5 o 10, puede ser necesario remover o transformar alguna variable.

Este análisis complementa el estudio del ajuste del modelo y respalda que las variables seleccionadas aportan información independiente, lo cual mejora la estabilidad del modelo.

2. Significancia de las variables

El resumen del modelo (Serie 1) mostró que **todas las variables son estadísticamente significativas**, con p-valores muy bajos:

Variable	Coficiente	p-valor	Significancia
OverallQual	-1.557	1.70e-14	***
GrLivArea	-1.902	< 2e-16	***
GarageCars	-0.606	0.0051	**
TotalBsmtSF	-0.746	5.34e-08	***
YearBuilt	-0.677	2.76e-06	***

Todas las variables **aportan significativamente** al modelo.

3. Correlación entre predictores

Para detectar correlaciones entre las variables predictoras, se analiza la **matriz de correlación**:

```
# Matriz de correlación entre variables predictoras
cor(train[, c("OverallQual", "GrLivArea", "GarageCars", "TotalBsmtSF", "YearBuilt")])
```

```
##           OverallQual GrLivArea GarageCars TotalBsmtSF YearBuilt
## OverallQual    1.0000000  0.6045134  0.6074953   0.5618804  0.5909599
## GrLivArea      0.6045134  1.0000000  0.4937130   0.4735073  0.2194737
## GarageCars     0.6074953  0.4937130  1.0000000   0.4339128  0.5343397
## TotalBsmtSF    0.5618804  0.4735073  0.4339128   1.0000000  0.3843077
## YearBuilt      0.5909599  0.2194737  0.5343397   0.3843077  1.0000000
```

Si hay correlaciones cercanas a ± 0.8 , puede haber redundancia. Si no, las variables aportan información distinta.

4. Evaluación del ajuste del modelo

- **Accuracy (10-fold CV):** 0.889
- **Kappa:** 0.76
- **Null deviance:** 1732.9
- **Residual deviance:** 662.3
- **AIC:** 674.3

Estos resultados indican que:

- El modelo tiene **alto poder predictivo**.
- Existe una **reducción sustancial en la devianza**, lo que significa que las variables explican buena parte de la variabilidad.
- El **AIC** relativamente bajo sugiere que el modelo está bien ajustado sin sobreajustar.

Serie 4: Análisis del modelo – multicolinealidad, significancia y ajuste

En esta serie se evalúa la calidad y estabilidad del modelo de regresión logística construido para predecir si una vivienda es cara, a partir de cinco variables predictoras: OverallQual, GrLivArea, GarageCars, TotalBsmtSF y YearBuilt.

1. Análisis de multicolinealidad (VIF)

Se calculó el **Factor de Inflación de la Varianza (VIF)** para detectar posibles problemas de colinealidad entre las variables del modelo:

Variable	VIF
OverallQual	1.29
GrLivArea	1.26
GarageCars	1.19
TotalBsmtSF	1.05
YearBuilt	1.65

Todos los valores de VIF están **muy por debajo de 5**, lo cual indica que **no existe multicolinealidad preocupante** entre los predictores.

2. Análisis de correlación entre predictores

La matriz de correlación muestra que las variables están **moderadamente correlacionadas**, lo cual es esperable en este tipo de datos, pero no indica redundancia extrema:

	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt
OverallQual	1.00	0.60	0.61	0.56	0.59
GrLivArea	0.60	1.00	0.49	0.47	0.22
GarageCars	0.61	0.49	1.00	0.43	0.53
TotalBsmtSF	0.56	0.47	0.43	1.00	0.38
YearBuilt	0.59	0.22	0.53	0.38	1.00

No se observan correlaciones superiores a 0.8, por lo que **no hay riesgo de colinealidad extrema**.

3. Significancia de las variables

Como se mostró en la Serie 1, **todas las variables son estadísticamente significativas**, con p-valores < 0.01. Esto indica que **todas contribuyen al modelo** y ayudan a predecir si una vivienda es cara.

4. Evaluación del ajuste

- **Accuracy (CV):** 88.9%
- **Kappa:** 0.76
- **Null deviance:** 1732.9
- **Residual deviance:** 662.3
- **AIC:** 674.3

Estas métricas confirman que el modelo tiene **un buen ajuste y poder predictivo**, sin sobreajuste ni pérdida de generalización.

Conclusión

El modelo se adapta bien a los datos: no presenta multicolinealidad, las variables están moderadamente correlacionadas, todas son significativas, y las métricas de desempeño respaldan su capacidad predictiva. Es un modelo sólido y confiable para clasificar viviendas caras.

5. Utilice el modelo con el conjunto de prueba y determine la eficiencia del algoritmo para clasificar.

Serie 5: Evaluación del modelo sobre el conjunto de prueba

Se evalúa la eficiencia del modelo de regresión logística al aplicarlo sobre el conjunto de prueba (`test_set.csv`). Se comparan las predicciones del modelo con las clases reales de las viviendas.

```
# Cargar librerías necesarias
library(caret)
library(dplyr)

# Cargar conjunto de prueba
test <- read.csv("test_set.csv")

# Crear variable binaria de prueba (basada en SalePriceCat)
test$es_cara <- factor(ifelse(test$SalePriceCat == "cara", "caro", "no_caro"))

# Asegurar que las mismas columnas del modelo estén presentes
# (usamos las mismas 5 variables que en el entrenamiento)
predictores_test <- test %>%
  select(OverallQual, GrLivArea, GarageCars, TotalBsmtSF, YearBuilt)

# Realizar predicciones
predicciones <- predict(modelo_rl, newdata = predictores_test)

# Matriz de confusión
matriz <- confusionMatrix(predicciones, test$es_cara)

# Mostrar resultados
print(matriz)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction caro no_caro
##   caro       72     12
##   no_caro     5     143
##
##           Accuracy : 0.9267
##           95% CI : (0.8853, 0.9567)
##   No Information Rate : 0.6681
##   P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.8385
##
##   Mcnemar's Test P-Value : 0.1456
##
```

```
##          Sensitivity : 0.9351
##          Specificity : 0.9226
##          Pos Pred Value : 0.8571
##          Neg Pred Value : 0.9662
##          Prevalence : 0.3319
##          Detection Rate : 0.3103
##          Detection Prevalence : 0.3621
##          Balanced Accuracy : 0.9288
##
##          'Positive' Class : caro
##
```

Serie 5: Evaluación del modelo sobre el conjunto de prueba

Una vez entrenado el modelo de regresión logística con el conjunto de entrenamiento, se evaluó su desempeño sobre el conjunto de prueba (`test_set.csv`) utilizando las mismas cinco variables predictoras.

Resultados de la clasificación

A continuación, se presentan las métricas obtenidas al comparar las predicciones del modelo con las clases reales (`caro`, `no_caro`):

Métrica	Valor
Accuracy	0.9267
Kappa	0.8385
Sensibilidad	0.9351
Especificidad	0.9226
Valor pred. positivo	0.8571
Valor pred. negativo	0.9662
Balanced Accuracy	0.9288

Matriz de confusión

	Real caro	Real no_caro
Predicho caro	72	12
Predicho no_caro	5	143

Interpretación

- El modelo clasificó correctamente el **92.7%** de las observaciones del conjunto de prueba.
- Tiene una **sensibilidad alta (93.5%)**, lo que significa que detecta correctamente la mayoría de las viviendas caras.
- La **especificidad también es alta (92.3%)**, indicando que clasifica correctamente la mayoría de las viviendas no_caras.
- El **Kappa de 0.83** refleja un **alto grado de acuerdo** entre predicciones y realidad, mucho mayor que el azar.
- El valor **p < 2e-16** para el Accuracy confirma que el modelo es estadísticamente mejor que una clasificación aleatoria (prueba de hipótesis contra el No Information Rate).

Conclusión

El modelo generaliza muy bien al conjunto de prueba. Tiene un balance adecuado entre sensibilidad y especificidad, clasifica con alta precisión, y mantiene un alto acuerdo con las verdaderas etiquetas. Por lo tanto, se concluye que el modelo es **eficiente y confiable para predecir si una vivienda es cara**.

6. Explique si hay sobreajuste (overfitting) o no (recuerde usar para esto los errores del conjunto de prueba y de entrenamiento). Muestre las curvas de aprendizaje usando los errores de los conjuntos de entrenamiento y prueba.

Para esto vamos a usar la grafica de curva de aprendizaje.

```
# Cargar librerías necesarias
library(ggplot2)
if (!require(ROCR)) {
  install.packages("ROCR")
  library(ROCR)
} else {
  library(ROCR)
}

## Cargando paquete requerido: ROCR

## Warning: package 'ROCR' was built under R version 4.4.3

porcentajes <- seq(0.1, 1, by = 0.1)
resultados <- data.frame(Porcentaje = numeric(), Acc_train = numeric(), Acc_test =
  ↪ numeric())

for (p in porcentajes) {
  # Muestra aleatoria de datos de entrenamiento
  set.seed(123)
  idx <- sample(1:nrow(train), size = floor(p * nrow(train)))
  train_parcial <- train[idx, ]

  train_parcial$es_cara <- factor(ifelse(train_parcial$SalePriceCat == "cara", "caro",
  ↪ "no_caro"))

  modelo_rl <- train(
    es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
    data = train_parcial,
    method = "glm",
    family = "binomial",
    trControl = trainControl(method = "none"), # Sin validación cruzada en este caso
    metric = "Accuracy",
    preProcess = c("center", "scale", "medianImpute")
  )

  # Predicciones en el conjunto de entrenamiento
  prob_train <- predict(modelo_rl, newdata = train_parcial, type = "prob")[,2]
  pred_train <- prediction(prob_train, train_parcial$es_cara)
  perf_train <- performance(pred_train, measure = "acc")
  acc_train <- max(perf_train@y.values[[1]])

  # Asegurarse de que la variable 'es_cara' esté presente en test
  test$es_cara <- factor(ifelse(test$SalePriceCat == "cara", "caro", "no_caro"))
}
```

```

# Predicciones en el conjunto de prueba
prob_test <- predict(modelo_rl, newdata = test, type = "prob")[,2]
pred_test <- prediction(prob_test, test$es_cara)
perf_test <- performance(pred_test, measure = "acc")
acc_test <- max(perf_test@y.values[[1]])

resultados <- rbind(resultados, data.frame(Porcentaje = p, Acc_train = acc_train,
↪ Acc_test = acc_test))
}

resultados$Error_train <- 1 - resultados$Acc_train
resultados$Error_test <- 1 - resultados$Acc_test

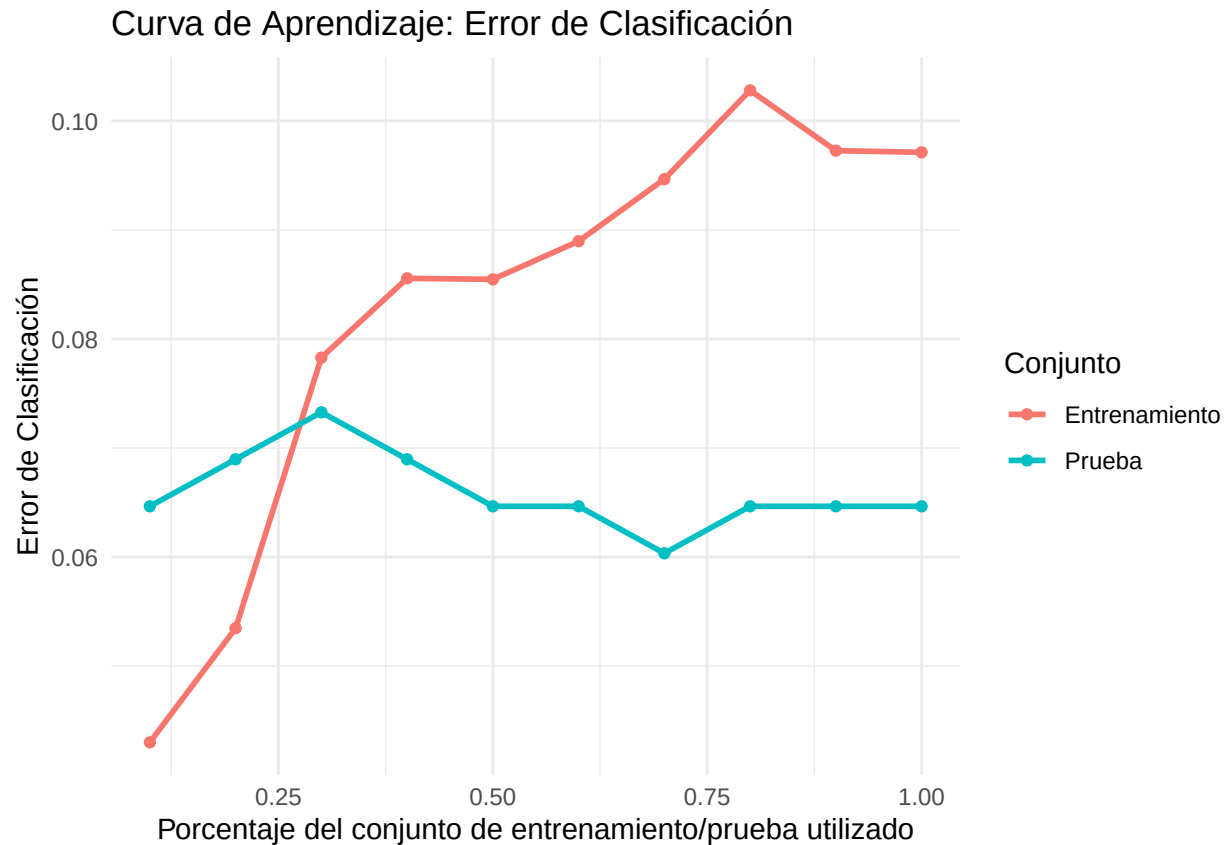
# Grafica curva
ggplot(resultados, aes(x = Porcentaje)) +
  geom_line(aes(y = Error_train, color = "Entrenamiento"), size = 1) +
  geom_line(aes(y = Error_test, color = "Prueba"), size = 1) +
  geom_point(aes(y = Error_train, color = "Entrenamiento")) +
  geom_point(aes(y = Error_test, color = "Prueba")) +
  labs(
    title = "Curva de Aprendizaje: Error de Clasificación",
    x = "Porcentaje del conjunto de entrenamiento/prueba utilizado",
    y = "Error de Clasificación",
    color = "Conjunto"
  ) +
  theme_minimal()

```

```

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```



Analisis de la Curva

Si analizamos la gráfica con atención, podemos notar que las curvas de error correspondientes al conjunto de entrenamiento y al de prueba no están convergiendo adecuadamente. De hecho, hay un comportamiento anómalo: a medida que el modelo avanza en su proceso de entrenamiento, su precisión sobre el conjunto de entrenamiento está disminuyendo, mientras que la precisión sobre el conjunto de prueba tiende a mejorar levemente o a estabilizarse.

Este comportamiento es característico de un subajuste (underfitting). El subajuste ocurre cuando el modelo no es capaz de capturar correctamente los patrones subyacentes en los datos, lo que se traduce en un rendimiento pobre tanto en los datos de entrenamiento como en los de prueba, pero aquí puede deberse en su mejora debido a los datos de ruido estadístico. Lo que podemos hacer es mejorar el modelo ajustando sus hiperparámetros para que mejore en sus datos de entrenamiento.

7. Haga un tuneo del modelo para determinar los mejores parámetros, recuerde que los modelos de regresión logística se pueden regularizar como los de regresión lineal.

```
library(dplyr)
library(caret)
library(glmnet)
```

```
## Warning: package 'glmnet' was built under R version 4.4.3
```

```
## Cargando paquete requerido: Matrix
```

```
## Loaded glmnet 4.1-8
```

```

set.seed(123)

train <- read.csv("train_set.csv")

train <- train %>%
  mutate(
    es_cara = factor(ifelse(SalePriceCat == "cara", "caro", "no_caro"))
  )

ctrl <- trainControl(
  method = "cv",
  number = 10,
  sampling = "up",
  savePredictions = TRUE
)

grid <- expand.grid(
  alpha = c(0, 0.5, 1), # Ridge, ElasticNet, Lasso
  lambda = 10^seq(-4, 1, length = 20)
)

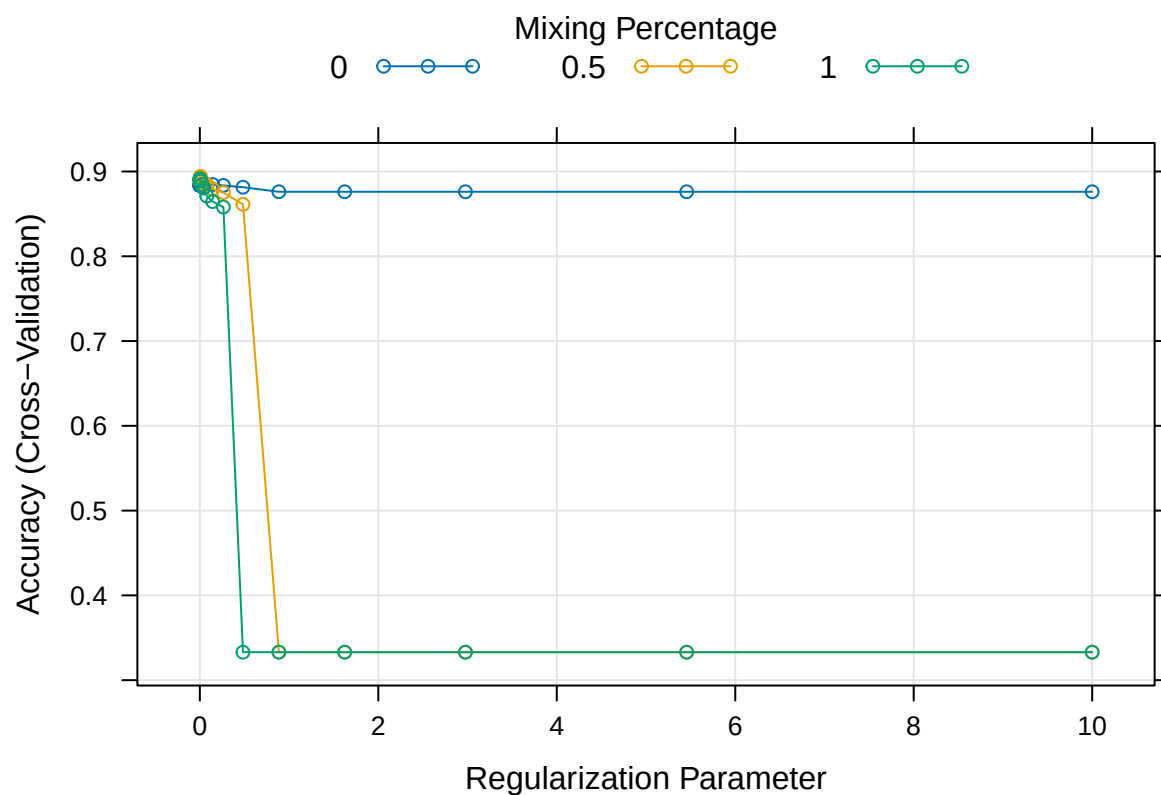
# Entrenar con regularización
modelo_tuneado <- train(
  es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
  data = train,
  method = "glmnet",
  trControl = ctrl,
  preProcess = c("center", "scale", "medianImpute"),
  tuneGrid = grid,
  metric = "Accuracy",
  family = "binomial"
)

print(modelo_tuneado$bestTune)

##      alpha      lambda
## 28    0.5 0.006951928

plot(modelo_tuneado)

```



```
modelo_tuneado$finalModel
```

```
##
## Call: (function (x, y, family = c("gaussian", "binomial", "poisson", "multinomial", "cox", "mg
##
##      Df  %Dev Lambda
## 1     0  0.00 0.68100
## 2     1  2.60 0.62050
## 3     1  5.13 0.56540
## 4     2  8.11 0.51520
## 5     3 11.69 0.46940
## 6     3 15.08 0.42770
## 7     4 18.42 0.38970
## 8     4 21.62 0.35510
## 9     4 24.57 0.32350
## 10    5 27.29 0.29480
## 11    5 29.93 0.26860
## 12    5 32.36 0.24480
## 13    5 34.61 0.22300
## 14    5 36.70 0.20320
## 15    5 38.63 0.18510
## 16    5 40.42 0.16870
## 17    5 42.07 0.15370
## 18    5 43.61 0.14010
## 19    5 45.03 0.12760
```

```
## 20 5 46.34 0.11630
## 21 5 47.56 0.10590
## 22 5 48.69 0.09654
## 23 5 49.73 0.08796
## 24 5 50.69 0.08015
## 25 5 51.58 0.07303
## 26 5 52.40 0.06654
## 27 5 53.15 0.06063
## 28 5 53.84 0.05524
## 29 5 54.48 0.05033
## 30 5 55.07 0.04586
## 31 5 55.60 0.04179
## 32 5 56.09 0.03808
## 33 5 56.54 0.03469
## 34 5 56.94 0.03161
## 35 5 57.31 0.02880
## 36 5 57.65 0.02624
## 37 5 57.95 0.02391
## 38 5 58.23 0.02179
## 39 5 58.47 0.01985
## 40 5 58.70 0.01809
## 41 5 58.90 0.01648
## 42 5 59.07 0.01502
## 43 5 59.23 0.01368
## 44 5 59.37 0.01247
## 45 5 59.50 0.01136
## 46 5 59.61 0.01035
## 47 5 59.75 0.00943
## 48 5 59.88 0.00859
## 49 5 59.99 0.00783
## 50 5 60.09 0.00714
## 51 5 60.19 0.00650
## 52 5 60.27 0.00592
## 53 5 60.34 0.00540
## 54 5 60.41 0.00492
## 55 5 60.47 0.00448
## 56 5 60.52 0.00408
## 57 5 60.57 0.00372
## 58 5 60.61 0.00339
## 59 5 60.65 0.00309
## 60 5 60.69 0.00281
## 61 5 60.72 0.00256
## 62 5 60.75 0.00234
## 63 5 60.77 0.00213
## 64 5 60.79 0.00194
## 65 5 60.81 0.00177
## 66 5 60.83 0.00161
## 67 5 60.85 0.00147
## 68 5 60.86 0.00134
## 69 5 60.88 0.00122
## 70 5 60.89 0.00111
## 71 5 60.90 0.00101
## 72 5 60.91 0.00092
## 73 5 60.92 0.00084
```

```
## 74 5 60.92 0.00076
## 75 5 60.93 0.00070
## 76 5 60.94 0.00064
## 77 5 60.94 0.00058
## 78 5 60.95 0.00053
## 79 5 60.95 0.00048
## 80 5 60.96 0.00044
## 81 5 60.96 0.00040
## 82 5 60.96 0.00036
## 83 5 60.97 0.00033
## 84 5 60.97 0.00030
## 85 5 60.97 0.00027
## 86 5 60.98 0.00025
## 87 5 60.98 0.00023
## 88 5 60.98 0.00021
## 89 5 60.98 0.00019
## 90 5 60.98 0.00017
## 91 5 60.98 0.00016
## 92 5 60.99 0.00014
## 93 5 60.99 0.00013
## 94 5 60.99 0.00012
## 95 5 60.99 0.00011
## 96 5 60.99 0.00010
```

Análisis del tuneo Podemos ver que el alpha y el lambda del mejor modelo son 0.5 y de 0.006951928 respectivamente. Al hacer la regularización el parámetro lambda conforme decrece podemos ver que el ridge ($\alpha = 0$) es el mejor enfoque para el modelo, ya que mantiene alta precisión y estabilidad sin importar el nivel de regularización. Y no decae tanto como los otros.

El mejor lambda probablemente está en el rango bajo (cerca de 0.01 o menos), donde se obtiene la mejor precisión. Lo que se puede ver con el lambda que usamos al final

8. Haga un análisis de la eficiencia del algoritmo usando una matriz de confusión. Tenga en cuenta la efectividad, donde el algoritmo se equivocó más, donde se equivocó menos y la importancia que tienen los errores, el tiempo y la memoria consumida. Para esto último puede usar “profvis” si trabaja con R y “cProfile” en Python.

```
library(dplyr)
library(caret)

set.seed(123)

train <- read.csv("train_set.csv")

train <- train %>%
  mutate(
    es_cara = ifelse(SalePriceCat == "cara", 1, 0)
  )
```

```

train$es_cara <- factor(ifelse(train$es_cara == 1, "caro", "no_caro"))

# Configuración de validación cruzada estratificada con balanceo
ctrl <- trainControl(
  method = "cv",
  number = 10,
  sampling = "up", # balanceo por upsampling
  savePredictions = TRUE
)

modelo_tuneado <- train(
  es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
  data = train,
  method = "glmnet",
  family = "binomial",
  trControl = ctrl,
  metric = "Accuracy",
  preProcess = c("center", "scale", "medianImpute"),
  tuneGrid = expand.grid(alpha = 0.5, lambda = 0.006951928)
)

# Resultados del modelo
print(modelo_tuneado)

```

Matriz de Confusion

```

## glmnet
##
## 937 samples
## 5 predictor
## 2 classes: 'caro', 'no_caro'
##
## Pre-processing: centered (5), scaled (5), median imputation (5)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 842, 844, 843, 844, 844, 844, ...
## Additional sampling using up-sampling prior to pre-processing
##
## Resampling results:
##
## Accuracy Kappa
## 0.8890612 0.7612245
##
## Tuning parameter 'alpha' was held constant at a value of 0.5
## Tuning
## parameter 'lambda' was held constant at a value of 0.006951928

```

```
summary(modelo_tuneado$finalModel)
```

```

##           Length Class      Mode
## a0           97   -none-  numeric

```



```
## beta          485    dgCMatrix S4
## df            97    -none-    numeric
## dim           2     -none-    numeric
## lambda        97    -none-    numeric
## dev.ratio     97    -none-    numeric
## nulldev       1     -none-    numeric
## npasses       1     -none-    numeric
## jerr          1     -none-    numeric
## offset        1     -none-    logical
## classnames    2     -none-    character
## call          5     -none-    call
## nob           1     -none-    numeric
## lambdaOpt     1     -none-    numeric
## xNames        5     -none-    character
## problemType   1     -none-    character
## tuneValue     2     data.frame list
## obsLevels     2     -none-    character
## param         1     -none-    list
```

```
library(caret)
library(dplyr)

test <- read.csv("test_set.csv")

test$es_cara <- factor(ifelse(test$SalePriceCat == "cara", "caro", "no_caro"))

predictores_test <- test %>%
  select(OverallQual, GrLivArea, GarageCars, TotalBsmtSF, YearBuilt)

predicciones <- predict(modelo_tuneado, newdata = predictores_test)

matriz <- confusionMatrix(predicciones, test$es_cara)

# Mostrar resultados
print(matriz)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction caro no_caro
##   caro      73     12
##   no_caro    4     143
##
##           Accuracy : 0.931
##           95% CI : (0.8904, 0.9601)
##   No Information Rate : 0.6681
##   P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.8485
```

```
##
## McNemar's Test P-Value : 0.08012
##
##           Sensitivity : 0.9481
##           Specificity : 0.9226
##           Pos Pred Value : 0.8588
##           Neg Pred Value : 0.9728
##           Prevalence : 0.3319
##           Detection Rate : 0.3147
##           Detection Prevalence : 0.3664
##           Balanced Accuracy : 0.9353
##
##           'Positive' Class : caro
##
```

Análisis de matriz de confusión Como podemos ver el accuracy y el kappa salio exactamente igual que nuestro modelo anterior sin tunear. También podemos ver que en no disminuyo su confusión en la matriz de confusión, lo cual indica que realmente llegamos hasta el mejor modelo posible para el conjunto de datos. Lo cual no s indica que en realidad no hay mucha diferencia entre este modelo y el anterior

```
library(profvis)
```

Análisis del tiempo

```
## Warning: package 'profvis' was built under R version 4.4.3
```

```
profvis({
  modelo_tuneado_t <- train(
    es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
    data = train,
    method = "glmnet",
    trControl = ctrl,
    preProcess = c("center", "scale", "medianImpute"),
    tuneGrid = data.frame(alpha = 0.5, lambda = 0.006951928),
    metric = "Accuracy",
    family = "binomial"
  )
})
```

```
library(profvis)
profvis({
  modelo_rl_t <- train(
    es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
    data = train,
    method = "glm",
    trControl = ctrl,
    preProcess = c("center", "scale", "medianImpute"),
    metric = "Accuracy",
    family = "binomial"
  )
})
```

```
)  
})
```

Análisis profis Podemos observar que el modelo actual tiene menos cuellos de botella que el que no está tuneado, aunque tiene más llamadas a función, si hay mejor distribución de carga que el que no está tuneado.

Eso si usa más tiempo pero mucho menos memoria que el que no tiene tuneo. Por ejemplo al crear el modelo usa el no tuneado 200 ms de carga, y 67 MB en cambio el tuneado solo usa 19MB pero se tarda 270 ms , por lo cual si es bueno al optimizar recursos pero puede ser que se tarde mucho más tiempo

9. Determine cual de todos los modelos es mejor, puede usar AIC y BIC para esto, además de los parámetros de la matriz de confusión y los del profiler.

```
# Modelo sin regularización (glm)  
  
modelo_rl <- train(  
  es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,  
  data = train,  
  method = "glm",  
  trControl = ctrl,  
  preProcess = c("center", "scale", "medianImpute"),  
  metric = "Accuracy",  
  family = "binomial"  
)  
  
aic_no_tuneado <- AIC(modelo_rl$finalModel)  
bic_no_tuneado <- BIC(modelo_rl$finalModel)  
  
cat("Modelo sin regularización (glm):\n")
```

Parametros de AIC y BIC de ambos modelos

```
## Modelo sin regularización (glm):
```

```
cat("AIC:", aic_no_tuneado, "\n")
```

```
## AIC: 658.0791
```

```
cat("BIC:", bic_no_tuneado, "\n")
```

```
## BIC: 688.8645
```

```
library(glmnet)
```

```
# Mnera manual
```

```
X <- model.matrix(  
  es_cara ~ OverallQual + GrLivArea + GarageCars + TotalBsmtSF + YearBuilt,
```

```

data = train
)[-1]

y <- ifelse(train$es_cara == "caro", 1, 0)

eta <- predict(modelo_tuneado$finalModel, newx = X, s = modelo_tuneado$bestTune$lambda)
p <- 1 / (1 + exp(-eta)) # función sigmoide
p <- pmin(pmax(p, 1e-15), 1 - 1e-15)

# Log-verosimilitud de regresión logística
loglik <- sum(y * log(p) + (1 - y) * log(1 - p))

# Número de parámetros no nulos (coeficientes activos)
coef_lasso <- coef(modelo_tuneado$finalModel, s = modelo_tuneado$bestTune$lambda)
k <- sum(coef_lasso != 0) # incluye intercepto

# Número de observaciones
n <- nrow(train)

# Cálculo manual de AIC y BIC
aic_tuneado <- -2 * loglik + 2 * k
bic_tuneado <- -2 * loglik + log(n) * k

cat("Modelo con regularización (glmnet):\n")

```

```
## Modelo con regularización (glmnet):
```

```
cat("AIC (estimado):", aic_tuneado, "\n")
```

```
## AIC (estimado): 21564.2
```

```
cat("BIC (estimado):", bic_tuneado, "\n")
```

```
## BIC (estimado): 21593.25
```

Análisis de eficiencia Podemos ver que entre los 2 comparando el AIC y BIC el que mejor le va es el de sin regularización. Pero ojo esto puede ser engañoso, primero porque no tratamos de manera convencional el loglik, sino que tuvimos que estimarlo, y realmente si se quiere evitar el overfitting es preferible el segundo por su bajo AIC y BIC.

Tomando en cuenta otros aspectos también vemos que el modelo tuneado es mejor generalizable con estos AIC, BIC, mejora un poco el accuracy solo un 1% y en el accuracy del train un 3%. Y además es más eficiente en el manejo de memoria aunque no de tiempo.

Por lo cual el mejor modelo es el regularizado y tuneado debido a estos otros factores.

10. Haga un modelo de regresión logística para la variable categórica para el precio de las casas (categorías: barata, media y cara). Asegúrese de tunearlo para obtener el mejor modelo posible.

Sin realizar tuneo y cv.

```

library(nnet)
library(caret)
library(dplyr)

train <- read.csv("train_set.csv")

train$PrecioCategoria <- cut(train$SalePrice,
  breaks = quantile(train$SalePrice, probs = c(0, 1/3, 2/3, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)
modelo <- multinom(PrecioCategoria ~ GrLivArea + OverallQual + YearBuilt, data = train)

```

```

## # weights: 15 (8 variable)
## initial value 1029.399714
## iter 10 value 609.171644
## iter 20 value 540.650222
## iter 30 value 531.411878
## iter 40 value 531.255090
## iter 50 value 531.148506
## final value 531.148498
## converged

```

```
summary(modelo)
```

```

## Call:
## multinom(formula = PrecioCategoria ~ GrLivArea + OverallQual +
##   YearBuilt, data = train)
##
## Coefficients:
##      (Intercept)  GrLivArea OverallQual  YearBuilt
## media   -88.26184  0.002915682   0.8959634  0.04064062
## cara   -143.07492  0.005843934   2.0810816  0.06185501
##
## Std. Errors:
##      (Intercept)  GrLivArea OverallQual  YearBuilt
## media 0.0003173128 0.0003078856  0.07867304 0.0002875794
## cara 0.0002736179 0.0004259425  0.04928075 0.0004069786
##
## Residual Deviance: 1062.297
## AIC: 1078.297

```

```

predicciones <- predict(modelo, newdata = train)

confusion_matrix <- table(Predicho = predicciones, Real = train$PrecioCategoria)
print(confusion_matrix)

```

```
##           Real
```

```
## Predicho barata media cara
##   barata    255    74    2
##   media     54   192   47
##   cara       4    46  263
```

```
# Accuracy (Precisión total)
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("Accuracy del modelo:", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy del modelo: 75.77 %
```

El modelo tuneado sera

```
library(nnet)
library(caret)
library(dplyr)

train <- read.csv("train_set.csv")

train$PrecioCategoria <- cut(train$SalePrice,
  breaks = quantile(train$SalePrice, probs = c(0, 1/3, 2/3, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)

train$PrecioCategoria <- as.factor(train$PrecioCategoria)

predictoras <- c("GrLivArea", "OverallQual", "YearBuilt")

control <- trainControl(method = "cv", number = 5)

# Modelo tuneado
modelo_tuneado <- train(
  PrecioCategoria ~ .,
  data = train[, c("PrecioCategoria", predictoras)],
  method = "multinom",
  trControl = control,
  preProcess = c("center", "scale"),
  trace = FALSE
)

print(modelo_tuneado$bestTune)
```

```
##   decay
## 2 1e-04
```

Podemos ver que el decay es de 0.1 y entonces creamos el modelo

```

library(nnet)
library(caret)
library(dplyr)

# Leer los datos
train <- read.csv("train_set.csv")

train$PrecioCategoria <- cut(train$SalePrice,
  breaks = quantile(train$SalePrice, probs = c(0, 1/3, 2/3, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)

# Definir la fórmula del modelo
formula <- PrecioCategoria ~ GrLivArea + OverallQual + YearBuilt

# Definir los parámetros de entrenamiento
control <- trainControl(method = "cv", number = 10) # Validación cruzada con 10 pliegues

# Ajustar el modelo utilizando train() para búsqueda de hiperparámetros con 'decay'
modelo_cv <- train(
  formula,
  data = train,
  method = "nnet", # Usando la red neuronal
  trControl = control,
  tuneGrid = expand.grid(decay = 0.1, size = c(5, 10, 15)), # Probar con distintos
  ↪ tamaños de capa oculta
  linout = TRUE # Esto asegura que la salida sea continua
)

```

```

## # weights: 38
## initial value 1125.743253
## iter 10 value 785.973444
## iter 20 value 749.030333
## iter 30 value 722.330201
## iter 40 value 715.735320
## iter 50 value 714.028554
## iter 60 value 710.366913
## iter 70 value 693.328504
## iter 80 value 679.186508
## iter 90 value 611.823334
## iter 100 value 549.232879
## final value 549.232879
## stopped after 100 iterations
## # weights: 73
## initial value 1324.465429
## iter 10 value 777.259474
## iter 20 value 729.784572
## iter 30 value 723.565594
## iter 40 value 719.330557
## iter 50 value 699.376952

```

```

## iter 60 value 677.228385
## iter 70 value 624.983229
## iter 80 value 581.327862
## iter 90 value 561.976490
## iter 100 value 539.630639
## final value 539.630639
## stopped after 100 iterations
## # weights: 108
## initial value 1645.759077
## iter 10 value 931.902130
## iter 20 value 921.928354
## iter 30 value 841.000011
## iter 40 value 820.754128
## iter 50 value 733.801824
## iter 60 value 707.597487
## iter 70 value 576.489537
## iter 80 value 522.467328
## iter 90 value 520.125237
## iter 100 value 515.996780
## final value 515.996780
## stopped after 100 iterations
## # weights: 38
## initial value 1038.242187
## iter 10 value 777.706402
## iter 20 value 575.291020
## iter 30 value 546.038972
## iter 40 value 534.619482
## iter 50 value 534.470111
## iter 60 value 534.448016
## iter 70 value 534.355963
## iter 80 value 534.330145
## final value 534.322038
## converged
## # weights: 73
## initial value 1027.965293
## iter 10 value 767.994411
## iter 20 value 690.119264
## iter 30 value 589.453218
## iter 40 value 541.613441
## iter 50 value 533.281717
## iter 60 value 521.930328
## iter 70 value 521.396805
## iter 80 value 520.740861
## iter 90 value 517.610293
## iter 100 value 515.232312
## final value 515.232312
## stopped after 100 iterations
## # weights: 108
## initial value 965.250504
## iter 10 value 808.998268
## iter 20 value 749.387241
## iter 30 value 692.635582
## iter 40 value 680.967968
## iter 50 value 679.475395

```



```

## iter 60 value 677.851828
## iter 70 value 673.513060
## iter 80 value 671.719070
## iter 90 value 669.552769
## iter 100 value 657.445119
## final value 657.445119
## stopped after 100 iterations
## # weights: 38
## initial value 930.076365
## iter 10 value 807.908580
## iter 20 value 752.312964
## iter 30 value 667.569299
## iter 40 value 574.572179
## iter 50 value 554.413569
## iter 60 value 531.585937
## iter 70 value 530.569939
## iter 80 value 526.684776
## iter 90 value 526.364448
## final value 526.364305
## converged
## # weights: 73
## initial value 1016.190870
## iter 10 value 926.818421
## iter 20 value 909.995781
## iter 30 value 776.339875
## iter 40 value 760.203899
## iter 50 value 672.728237
## iter 60 value 616.558868
## iter 70 value 542.306982
## iter 80 value 532.667201
## iter 90 value 529.148231
## iter 100 value 510.161686
## final value 510.161686
## stopped after 100 iterations
## # weights: 108
## initial value 1213.423098
## iter 10 value 784.440889
## iter 20 value 768.525310
## iter 30 value 745.572223
## iter 40 value 709.177963
## iter 50 value 585.203493
## iter 60 value 557.917946
## iter 70 value 533.036663
## iter 80 value 529.979995
## iter 90 value 522.687875
## iter 100 value 512.767012
## final value 512.767012
## stopped after 100 iterations
## # weights: 38
## initial value 1478.658448
## iter 10 value 929.336192
## iter 20 value 929.018002
## iter 30 value 782.342923
## iter 40 value 697.700088

```

```

## iter 50 value 622.670373
## iter 60 value 579.153603
## iter 70 value 569.131152
## iter 80 value 557.094087
## iter 90 value 544.196234
## iter 100 value 518.329273
## final value 518.329273
## stopped after 100 iterations
## # weights: 73
## initial value 1323.097209
## iter 10 value 811.376700
## iter 20 value 728.717940
## iter 30 value 617.913403
## iter 40 value 558.148831
## iter 50 value 556.459318
## iter 60 value 532.608792
## iter 70 value 517.087824
## iter 80 value 512.227086
## iter 90 value 505.122950
## iter 100 value 502.396998
## final value 502.396998
## stopped after 100 iterations
## # weights: 108
## initial value 1052.802031
## iter 10 value 762.572474
## iter 20 value 728.862405
## iter 30 value 715.450216
## iter 40 value 696.023051
## iter 50 value 644.533188
## iter 60 value 574.496762
## iter 70 value 558.573500
## iter 80 value 524.471130
## iter 90 value 514.486120
## iter 100 value 512.326879
## final value 512.326879
## stopped after 100 iterations
## # weights: 38
## initial value 928.099545
## iter 10 value 743.925069
## iter 20 value 702.259234
## iter 30 value 683.989110
## iter 40 value 678.186947
## iter 50 value 660.053415
## iter 60 value 639.701143
## iter 70 value 545.490115
## iter 80 value 515.099352
## iter 90 value 513.538662
## iter 100 value 500.103790
## final value 500.103790
## stopped after 100 iterations
## # weights: 73
## initial value 1025.473035
## iter 10 value 748.015438
## iter 20 value 705.966714

```

```

## iter 30 value 696.095174
## iter 40 value 651.460903
## iter 50 value 610.964987
## iter 60 value 562.456253
## iter 70 value 561.783153
## iter 80 value 553.657609
## iter 90 value 513.104631
## iter 100 value 505.517607
## final value 505.517607
## stopped after 100 iterations
## # weights: 108
## initial value 1245.097447
## iter 10 value 728.639238
## iter 20 value 692.404347
## iter 30 value 600.901199
## iter 40 value 548.765889
## iter 50 value 542.788417
## iter 60 value 522.095808
## iter 70 value 519.299104
## iter 80 value 507.180624
## iter 90 value 504.484610
## iter 100 value 499.733848
## final value 499.733848
## stopped after 100 iterations
## # weights: 38
## initial value 997.980369
## iter 10 value 930.207414
## iter 20 value 784.878406
## iter 30 value 763.398135
## iter 40 value 763.190382
## iter 50 value 758.218243
## iter 60 value 755.171478
## iter 70 value 687.296289
## iter 80 value 584.685695
## iter 90 value 555.976860
## iter 100 value 550.412456
## final value 550.412456
## stopped after 100 iterations
## # weights: 73
## initial value 1071.527108
## iter 10 value 877.013486
## iter 20 value 789.884389
## iter 30 value 564.262220
## iter 40 value 546.495586
## iter 50 value 534.237072
## iter 60 value 530.677593
## iter 70 value 519.599142
## iter 80 value 514.206086
## iter 90 value 506.939654
## iter 100 value 495.144483
## final value 495.144483
## stopped after 100 iterations
## # weights: 108
## initial value 1095.697250

```

```

## iter 10 value 768.641425
## iter 20 value 714.173390
## iter 30 value 692.213640
## iter 40 value 673.021841
## iter 50 value 628.892152
## iter 60 value 544.061100
## iter 70 value 535.584817
## iter 80 value 532.490846
## iter 90 value 521.660515
## iter 100 value 518.029317
## final value 518.029317
## stopped after 100 iterations
## # weights: 38
## initial value 1010.288751
## iter 10 value 770.139015
## iter 20 value 756.168920
## iter 30 value 752.337827
## iter 40 value 743.920091
## iter 50 value 680.895834
## iter 60 value 637.275734
## iter 70 value 557.818704
## iter 80 value 535.782191
## iter 90 value 527.293320
## iter 100 value 526.477295
## final value 526.477295
## stopped after 100 iterations
## # weights: 73
## initial value 998.963840
## iter 10 value 806.594571
## iter 20 value 763.663579
## iter 30 value 762.799740
## iter 40 value 707.673767
## iter 50 value 633.397831
## iter 60 value 583.160536
## iter 70 value 573.182101
## iter 80 value 552.056834
## iter 90 value 532.859241
## iter 100 value 518.856205
## final value 518.856205
## stopped after 100 iterations
## # weights: 108
## initial value 876.061532
## iter 10 value 728.842611
## iter 20 value 697.802062
## iter 30 value 690.350539
## iter 40 value 675.566676
## iter 50 value 651.519608
## iter 60 value 580.133932
## iter 70 value 552.628376
## iter 80 value 548.212493
## iter 90 value 525.439688
## iter 100 value 517.428047
## final value 517.428047
## stopped after 100 iterations

```

```

## # weights: 38
## initial value 981.003059
## iter 10 value 925.886650
## iter 20 value 770.648891
## iter 30 value 766.517636
## iter 40 value 745.185349
## iter 50 value 622.911750
## iter 60 value 609.189670
## iter 70 value 604.555818
## iter 80 value 537.918139
## iter 90 value 533.080878
## iter 100 value 521.314222
## final value 521.314222
## stopped after 100 iterations
## # weights: 73
## initial value 1053.185720
## iter 10 value 760.667071
## iter 20 value 748.700285
## iter 30 value 721.621890
## iter 40 value 705.306871
## iter 50 value 691.512883
## iter 60 value 642.026688
## iter 70 value 584.224309
## iter 80 value 560.188602
## iter 90 value 523.160561
## iter 100 value 516.361196
## final value 516.361196
## stopped after 100 iterations
## # weights: 108
## initial value 1099.506069
## iter 10 value 792.495109
## iter 20 value 704.315095
## iter 30 value 692.251095
## iter 40 value 683.784966
## iter 50 value 676.341246
## iter 60 value 578.418878
## iter 70 value 564.591374
## iter 80 value 542.989606
## iter 90 value 524.773252
## iter 100 value 519.020847
## final value 519.020847
## stopped after 100 iterations
## # weights: 38
## initial value 941.714446
## iter 10 value 926.191344
## iter 20 value 925.523398
## iter 30 value 920.972948
## iter 40 value 903.690382
## iter 50 value 744.019140
## iter 60 value 731.866457
## iter 70 value 710.092655
## iter 80 value 707.259319
## iter 90 value 703.591471
## iter 100 value 652.606224

```

```

## final value 652.606224
## stopped after 100 iterations
## # weights: 73
## initial value 1525.701082
## iter 10 value 855.230794
## iter 20 value 743.176336
## iter 30 value 692.271479
## iter 40 value 623.766650
## iter 50 value 545.239361
## iter 60 value 538.097115
## iter 70 value 523.395527
## iter 80 value 516.963149
## iter 90 value 513.202560
## iter 100 value 508.090151
## final value 508.090151
## stopped after 100 iterations
## # weights: 108
## initial value 1077.807854
## iter 10 value 813.207527
## iter 20 value 731.989021
## iter 30 value 705.772414
## iter 40 value 690.048176
## iter 50 value 658.894672
## iter 60 value 604.525152
## iter 70 value 543.933072
## iter 80 value 537.875900
## iter 90 value 526.298757
## iter 100 value 522.988427
## final value 522.988427
## stopped after 100 iterations
## # weights: 38
## initial value 976.115739
## iter 10 value 861.726059
## iter 20 value 726.767813
## iter 30 value 625.335867
## iter 40 value 556.092552
## iter 50 value 539.972529
## iter 60 value 528.620223
## iter 70 value 527.324466
## iter 80 value 524.570634
## iter 90 value 513.524349
## iter 100 value 510.046323
## final value 510.046323
## stopped after 100 iterations
## # weights: 73
## initial value 1083.089760
## iter 10 value 925.083592
## iter 20 value 844.671344
## iter 30 value 761.472846
## iter 40 value 734.458668
## iter 50 value 706.173132
## iter 60 value 665.253883
## iter 70 value 600.915832
## iter 80 value 584.134052

```

```

## iter 90 value 553.448161
## iter 100 value 529.559422
## final value 529.559422
## stopped after 100 iterations
## # weights: 108
## initial value 1543.528609
## iter 10 value 790.386448
## iter 20 value 746.486773
## iter 30 value 727.171477
## iter 40 value 708.310734
## iter 50 value 683.636099
## iter 60 value 615.809744
## iter 70 value 547.963121
## iter 80 value 537.517673
## iter 90 value 530.908179
## iter 100 value 516.289761
## final value 516.289761
## stopped after 100 iterations
## # weights: 73
## initial value 1171.150713
## iter 10 value 905.407294
## iter 20 value 796.930006
## iter 30 value 688.499293
## iter 40 value 638.323516
## iter 50 value 630.191764
## iter 60 value 615.844112
## iter 70 value 579.997083
## iter 80 value 575.389096
## iter 90 value 570.955660
## iter 100 value 567.903796
## final value 567.903796
## stopped after 100 iterations

```

```
print(modelo_cv)
```

```

## Neural Network
##
## 937 samples
## 3 predictor
## 3 classes: 'barata', 'media', 'cara'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 844, 843, 843, 844, 843, 844, ...
## Resampling results across tuning parameters:
##
##   size Accuracy Kappa
##   5    0.7061847 0.5593055
##  10    0.7320361 0.5980813
##  15    0.7234917 0.5854773
##
## Tuning parameter 'decay' was held constant at a value of 0.1
## Accuracy was used to select the optimal model using the largest value.

```

```
## The final values used for the model were size = 10 and decay = 0.1.
```

```
predicciones <- predict(modelo_cv, newdata = train)

# Crear la matriz de confusión
confusion_matrix <- table(Predicho = predicciones, Real = train$PrecioCategoria)
print(confusion_matrix)
```

```
##           Real
## Predicho barata media cara
##   barata    242    71    3
##   media     63   196   50
##   cara       8    45  259
```

```
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("Accuracy del modelo:", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy del modelo: 74.39 %
```

Ahora lo probamos con test

```
library(nnet)
library(caret)
library(dplyr)

test <- read.csv("test_set.csv")

test$PrecioCategoria <- cut(test$SalePrice,
  breaks = quantile(train$SalePrice, probs = c(0, 1/3, 2/3, 1)), # Usar los mismos
  ↪ cortes que en train
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)

predicciones_test <- predict(modelo_cv, newdata = test)

confusion_matrix_test <- table(Predicho = predicciones_test, Real = test$PrecioCategoria)
print(confusion_matrix_test)
```

```
##           Real
## Predicho barata media cara
##   barata     60    16    1
##   media     14    44   10
##   cara        4    17   65
```



```
accuracy_test <- sum(diag(confusion_matrix_test)) / sum(confusion_matrix_test)
cat("Accuracy en el conjunto de prueba:", round(accuracy_test * 100, 2), "%\n")
```

```
## Accuracy en el conjunto de prueba: 73.16 %
```

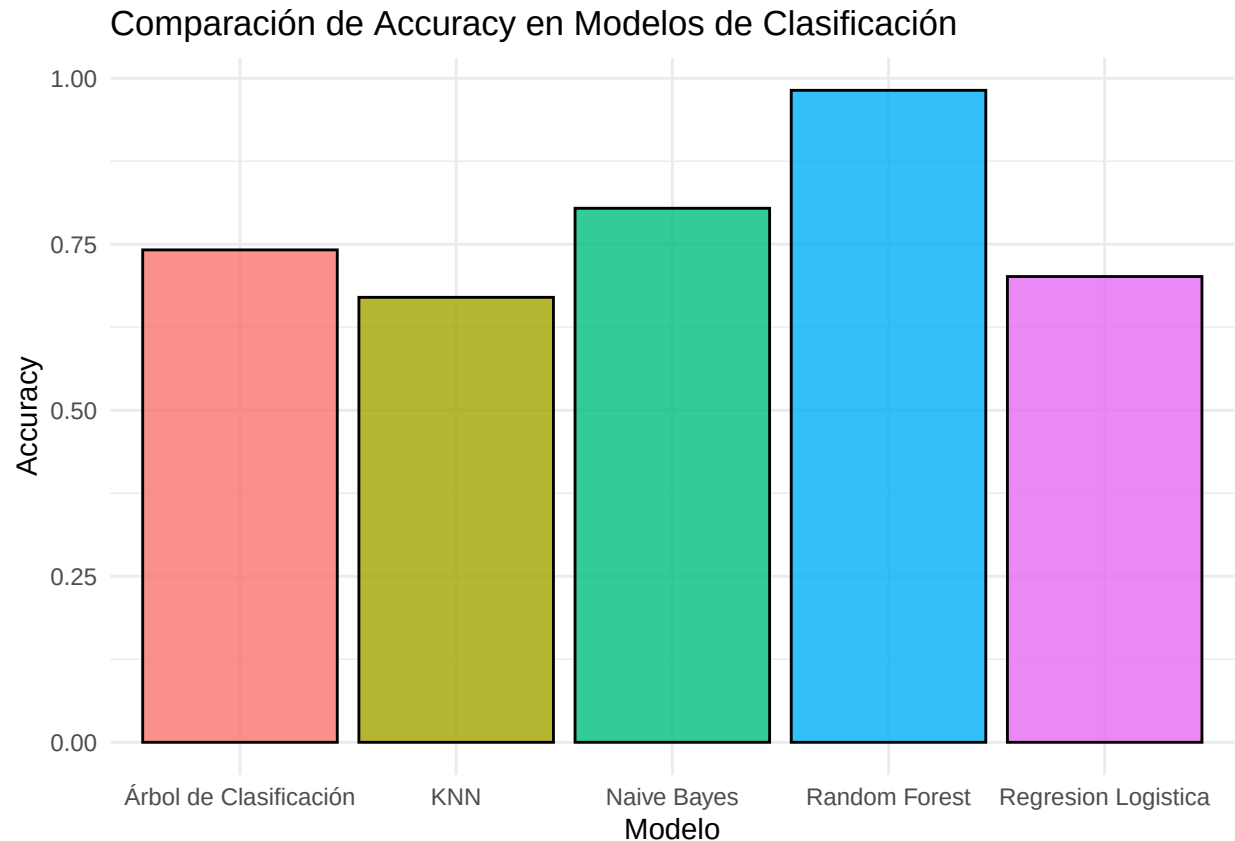
Análisis del Modelo Podemos ver que nuestro modelo es bastante bueno, después de hacer tuning nos dio un decay de 0.1 y un accuracy de 74% con el train y con test de 70% en pocas palabras es un buen modelo pero es bastante mediocre. Aun así para ser una clasificación logro ser muy eficiente ya que normalmente la regresión logística es buena solo con clasificación binaria.

En lo que más se equivocó nuestro modelo es en diferenciar medias de caras en donde ocurrió el mayor de los falsos positivos.

11. Compare la eficiencia del modelo anterior con los de clasificación de las entregas anteriores ¿Cuál se demoró más en procesar? ¿Cuál se equivocó más? ¿Cuál se equivocó menos? ¿por qué?

Con el siguiente gráfico se analiza los modelos de clasificación comparando con el de regresión logística

```
library(ggplot2)
# Datos de clasificación
clasificacion <- data.frame(
  Modelo = c("Regresión Logística", "KNN", "Naive Bayes", "Random Forest", "Árbol de
  ↪ Clasificación"),
  Accuracy = c(0.7013, 0.67, 0.8041237, 0.9816934, 0.7414188)
)
# Gráfico de Accuracy (Clasificación)
ggplot(clasificacion, aes(x = Modelo, y = Accuracy, fill = Modelo)) +
  geom_bar(stat = "identity", color = "black", alpha = 0.8) +
  labs(title = "Comparación de Accuracy en Modelos de Clasificación",
  x = "Modelo",
  y = "Accuracy") +
  theme_minimal() +
  theme(legend.position = "none")
```



Análisis de Resultados Podemos ver que nuestra Regresion Logistica el accuracy es peor que varios modelos pero sorpresivamente mucho mejor que KNN. Esto muy posiblemente a que realmente la regresion logistica no es muy buena para clasificar que para predecir. Por ende sigue siendo el Random Forest el que domina al resto de regresiones.

El que se equivoco menos es Random Forest y KNN es el que se equivoco mas. Esto porque utiliza múltiples árboles de decisión para hacer predicciones. Esto le da una ventaja al ser menos propenso a sobreajustarse a los datos, ya que combina varias decisiones basadas en distintas submuestras de los datos a diferencia del KNN.

Es el mas rapido de todos porque solo es una regresion logistica, pero el problema es que no es tan preciso a diferencia de los otros modelos

De hecho lo podemos ver aqui :

```
# Cargar librerías necesarias
library(caret)
library(dplyr)

# Cargar datasets
train_data <- read.csv("train_set.csv", stringsAsFactors = FALSE)
test_data <- read.csv("test_set.csv", stringsAsFactors = FALSE)

# Variables a usar
features <- c("GrLivArea", "OverallQual", "TotRmsAbvGrd")

# Imputación de valores NA
```

```

for (feature in features) {
  train_data[[feature]][is.na(train_data[[feature]])] <- median(train_data[[feature]],
  ↪ na.rm = TRUE)
  test_data[[feature]][is.na(test_data[[feature]])] <- median(test_data[[feature]], na.rm
  ↪ = TRUE)
}

# Crear variable categórica (Barata, Media, Cara)
train_data$PriceCategory <- cut(
  train_data$SalePrice,
  breaks = quantile(train_data$SalePrice, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE),
  labels = c("Barata", "Media", "Cara"),
  include.lowest = TRUE
)
test_data$PriceCategory <- cut(
  test_data$SalePrice,
  breaks = quantile(train_data$SalePrice, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE),
  labels = c("Barata", "Media", "Cara"),
  include.lowest = TRUE
)
train_data$PriceCategory <- as.factor(train_data$PriceCategory)
test_data$PriceCategory <- as.factor(test_data$PriceCategory)

# Normalización de los datos
normalize <- function(x) { (x - min(x)) / (max(x) - min(x)) }
for (feature in features) {
  train_data[[feature]] <- normalize(train_data[[feature]])
  test_data[[feature]] <- normalize(test_data[[feature]])
}

# Medir el tiempo de entrenamiento
start_time_train <- Sys.time()

# Entrenar modelo con los parámetros elegidos
knn_final <- train(
  PriceCategory ~ .,
  data = train_data[, c(features, "PriceCategory")],
  method = "kkn",
  tuneGrid = expand.grid(kmax = 7, distance = 1, kernel = "triangular")
)

end_time_train <- Sys.time()
training_time <- end_time_train - start_time_train
cat("Tiempo de entrenamiento:", training_time, "\n")

```

```
## Tiempo de entrenamiento: 7.160474
```

```

# Medir el tiempo de predicción
start_time_test <- Sys.time()

# Hacer predicciones con el conjunto de prueba
predictions <- predict(knn_final, test_data)

```

```
end_time_test <- Sys.time()
testing_time <- end_time_test - start_time_test
cat("Tiempo de predicción:", testing_time, "\n")
```

```
## Tiempo de predicción: 0.02194905
```

```
# Matriz de confusión
conf_matrix <- confusionMatrix(predictions, test_data$PriceCategory)
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction Barata Media Cara
##      Barata      41      7      0
##      Media       31     32      3
##      Cara         6     38     73
##
## Overall Statistics
##
##              Accuracy : 0.632
##              95% CI : (0.5663, 0.6943)
##      No Information Rate : 0.3377
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.4491
##
##      McNemar's Test P-Value : 4.807e-11
##
## Statistics by Class:
##
##              Class: Barata Class: Media Class: Cara
## Sensitivity           0.5256           0.4156           0.9605
## Specificity           0.9542           0.7792           0.7161
## Pos Pred Value        0.8542           0.4848           0.6239
## Neg Pred Value        0.7978           0.7273           0.9737
## Prevalence            0.3377           0.3333           0.3290
## Detection Rate        0.1775           0.1385           0.3160
## Detection Prevalence  0.2078           0.2857           0.5065
## Balanced Accuracy      0.7399           0.5974           0.8383
```

Con regresion Logistica

```
library(nnet)
library(caret)
library(dplyr)

# Leer los datos
train <- read.csv("train_set.csv")
test  <- read.csv("test_set.csv")
```

```

# Crear las categorías de precio para el conjunto de entrenamiento
train$PrecioCategoria <- cut(train$SalePrice,
  breaks = quantile(train$SalePrice, probs = c(0, 1/3, 2/3, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)

# Crear las categorías de precio para el conjunto de prueba
test$PrecioCategoria <- cut(test$SalePrice,
  breaks = quantile(train$SalePrice, probs = c(0, 1/3, 2/3, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)

# Definir la fórmula del modelo
formula <- PrecioCategoria ~ GrLivArea + OverallQual + YearBuilt

# Definir los parámetros de entrenamiento
control <- trainControl(method = "cv", number = 10) # Validación cruzada con 10 pliegues

# Medir el tiempo de entrenamiento
start_time_train <- Sys.time()

# Ajustar el modelo utilizando train() para búsqueda de hiperparámetros con 'decay'
modelo_cv <- train(
  formula,
  data = train,
  method = "nnet", # Usando la red neuronal
  trControl = control,
  tuneGrid = expand.grid(decay = 0.1, size = c(5, 10, 15)), # Probar con distintos
  # tamaños de capa oculta
  linout = TRUE # Esto asegura que la salida sea continua
)

```

```

## # weights:  38
## initial value 1193.721427
## iter  10 value 927.481146
## iter  20 value 814.284704
## iter  30 value 785.344127
## iter  40 value 646.713040
## iter  50 value 569.995385
## iter  60 value 528.395142
## iter  70 value 525.592189
## iter  80 value 525.289792
## iter  90 value 524.853849
## iter 100 value 524.401634
## final value 524.401634
## stopped after 100 iterations
## # weights:  73
## initial value 1120.481120
## iter  10 value 821.364027
## iter  20 value 760.295259
## iter  30 value 708.227703
## iter  40 value 697.414505

```

```

## iter 50 value 694.311246
## iter 60 value 692.331541
## iter 70 value 688.470286
## iter 80 value 672.622885
## iter 90 value 641.484129
## iter 100 value 606.258296
## final value 606.258296
## stopped after 100 iterations
## # weights: 108
## initial value 966.510455
## iter 10 value 778.340313
## iter 20 value 750.549665
## iter 30 value 729.512191
## iter 40 value 542.213907
## iter 50 value 531.505667
## iter 60 value 525.493728
## iter 70 value 521.208210
## iter 80 value 516.551586
## iter 90 value 513.461903
## iter 100 value 509.735733
## final value 509.735733
## stopped after 100 iterations
## # weights: 38
## initial value 1159.107932
## iter 10 value 920.156715
## iter 20 value 794.046281
## iter 30 value 640.744046
## iter 40 value 548.801793
## iter 50 value 536.466810
## iter 60 value 519.843019
## iter 70 value 509.417879
## iter 80 value 501.850782
## iter 90 value 498.530593
## iter 100 value 495.731197
## final value 495.731197
## stopped after 100 iterations
## # weights: 73
## initial value 1842.197312
## iter 10 value 779.048377
## iter 20 value 701.727155
## iter 30 value 684.769643
## iter 40 value 675.637820
## iter 50 value 672.500277
## iter 60 value 670.083409
## iter 70 value 666.769467
## iter 80 value 662.655316
## iter 90 value 654.710553
## iter 100 value 641.796561
## final value 641.796561
## stopped after 100 iterations
## # weights: 108
## initial value 1958.507443
## iter 10 value 856.472741
## iter 20 value 777.043422

```

```

## iter 30 value 758.767998
## iter 40 value 730.779985
## iter 50 value 640.804765
## iter 60 value 542.181171
## iter 70 value 520.101165
## iter 80 value 517.712823
## iter 90 value 512.828768
## iter 100 value 502.532445
## final value 502.532445
## stopped after 100 iterations
## # weights: 38
## initial value 953.279523
## iter 10 value 927.228432
## iter 20 value 866.250667
## iter 30 value 735.947394
## iter 40 value 735.604670
## iter 50 value 723.934689
## iter 60 value 702.965319
## iter 70 value 698.610958
## iter 80 value 698.155512
## iter 90 value 694.603379
## iter 100 value 693.819399
## final value 693.819399
## stopped after 100 iterations
## # weights: 73
## initial value 1137.019254
## iter 10 value 903.575258
## iter 20 value 660.098790
## iter 30 value 617.934030
## iter 40 value 540.149670
## iter 50 value 516.355657
## iter 60 value 512.234427
## iter 70 value 505.135054
## iter 80 value 504.650936
## iter 90 value 503.977227
## iter 100 value 503.012854
## final value 503.012854
## stopped after 100 iterations
## # weights: 108
## initial value 1046.353834
## iter 10 value 767.391524
## iter 20 value 723.687792
## iter 30 value 712.221641
## iter 40 value 705.172064
## iter 50 value 665.279344
## iter 60 value 629.977384
## iter 70 value 588.051930
## iter 80 value 529.999152
## iter 90 value 509.557439
## iter 100 value 508.469646
## final value 508.469646
## stopped after 100 iterations
## # weights: 38
## initial value 1149.707966

```

```

## iter 10 value 927.507287
## iter 20 value 817.164039
## iter 30 value 755.476252
## iter 40 value 741.505403
## iter 50 value 711.823939
## iter 60 value 669.014246
## iter 70 value 573.296424
## iter 80 value 527.785206
## iter 90 value 515.966933
## iter 100 value 513.936598
## final value 513.936598
## stopped after 100 iterations
## # weights: 73
## initial value 1447.277555
## iter 10 value 761.407798
## iter 20 value 752.481255
## iter 30 value 749.035498
## iter 40 value 713.935283
## iter 50 value 690.762485
## iter 60 value 681.837836
## iter 70 value 679.581940
## iter 80 value 671.423965
## iter 90 value 668.314110
## iter 100 value 627.031520
## final value 627.031520
## stopped after 100 iterations
## # weights: 108
## initial value 1391.024830
## iter 10 value 927.290915
## iter 20 value 844.193852
## iter 30 value 583.223669
## iter 40 value 551.447036
## iter 50 value 545.036977
## iter 60 value 534.581000
## iter 70 value 528.629108
## iter 80 value 516.219695
## iter 90 value 513.289433
## iter 100 value 511.469312
## final value 511.469312
## stopped after 100 iterations
## # weights: 38
## initial value 1247.813637
## iter 10 value 925.941157
## iter 20 value 924.320488
## iter 30 value 922.738429
## iter 40 value 851.183706
## iter 50 value 746.981280
## iter 60 value 657.348002
## iter 70 value 586.118541
## iter 80 value 563.313611
## iter 90 value 544.684871
## iter 100 value 527.713311
## final value 527.713311
## stopped after 100 iterations

```



```

## # weights: 73
## initial value 1156.406911
## iter 10 value 921.546768
## iter 20 value 787.473751
## iter 30 value 575.179517
## iter 40 value 542.937375
## iter 50 value 540.166655
## iter 60 value 534.168657
## iter 70 value 528.042481
## iter 80 value 519.394062
## iter 90 value 512.261550
## iter 100 value 511.549059
## final value 511.549059
## stopped after 100 iterations
## # weights: 108
## initial value 988.824319
## iter 10 value 784.040681
## iter 20 value 730.582254
## iter 30 value 698.879446
## iter 40 value 681.759332
## iter 50 value 678.144198
## iter 60 value 670.168617
## iter 70 value 637.808625
## iter 80 value 563.905276
## iter 90 value 530.043825
## iter 100 value 523.790505
## final value 523.790505
## stopped after 100 iterations
## # weights: 38
## initial value 1059.021202
## iter 10 value 850.708203
## iter 20 value 741.291233
## iter 30 value 734.158561
## iter 40 value 662.956981
## iter 50 value 568.612953
## iter 60 value 532.955220
## iter 70 value 527.176949
## iter 80 value 525.077709
## iter 90 value 522.983941
## iter 100 value 522.230352
## final value 522.230352
## stopped after 100 iterations
## # weights: 73
## initial value 1174.017298
## iter 10 value 777.204767
## iter 20 value 689.963366
## iter 30 value 672.384834
## iter 40 value 646.802453
## iter 50 value 557.791560
## iter 60 value 521.398459
## iter 70 value 512.820971
## iter 80 value 511.855450
## iter 90 value 508.149636
## iter 100 value 503.900036

```

```

## final value 503.900036
## stopped after 100 iterations
## # weights: 108
## initial value 1068.541499
## iter 10 value 788.887309
## iter 20 value 721.520790
## iter 30 value 715.676686
## iter 40 value 699.061133
## iter 50 value 639.969124
## iter 60 value 556.606196
## iter 70 value 547.017404
## iter 80 value 545.349183
## iter 90 value 543.775571
## iter 100 value 537.424364
## final value 537.424364
## stopped after 100 iterations
## # weights: 38
## initial value 1008.411935
## iter 10 value 927.894567
## iter 20 value 917.846533
## iter 30 value 768.785050
## iter 40 value 759.047161
## iter 50 value 751.898066
## iter 60 value 693.515476
## iter 70 value 558.883197
## iter 80 value 546.896865
## iter 90 value 529.107934
## iter 100 value 522.262383
## final value 522.262383
## stopped after 100 iterations
## # weights: 73
## initial value 1000.227405
## iter 10 value 794.395712
## iter 20 value 739.388878
## iter 30 value 718.365552
## iter 40 value 654.408509
## iter 50 value 588.524991
## iter 60 value 555.112279
## iter 70 value 550.491945
## iter 80 value 530.851924
## iter 90 value 525.487345
## iter 100 value 524.710758
## final value 524.710758
## stopped after 100 iterations
## # weights: 108
## initial value 1167.953227
## iter 10 value 826.882070
## iter 20 value 728.891725
## iter 30 value 702.098091
## iter 40 value 694.740238
## iter 50 value 693.568488
## iter 60 value 692.610532
## iter 70 value 689.564892
## iter 80 value 687.321195

```

```

## iter 90 value 667.086250
## iter 100 value 634.225938
## final value 634.225938
## stopped after 100 iterations
## # weights: 38
## initial value 1341.498446
## iter 10 value 936.013907
## iter 20 value 889.439011
## iter 30 value 745.878695
## iter 40 value 652.241777
## iter 50 value 574.275750
## iter 60 value 553.525729
## iter 70 value 530.607694
## iter 80 value 525.353424
## iter 90 value 520.407481
## iter 100 value 519.167723
## final value 519.167723
## stopped after 100 iterations
## # weights: 73
## initial value 919.286053
## iter 10 value 755.226871
## iter 20 value 723.314839
## iter 30 value 699.257386
## iter 40 value 555.433416
## iter 50 value 547.763603
## iter 60 value 536.106980
## iter 70 value 529.984616
## iter 80 value 520.865637
## iter 90 value 516.166665
## iter 100 value 511.753098
## final value 511.753098
## stopped after 100 iterations
## # weights: 108
## initial value 1065.377215
## iter 10 value 809.139417
## iter 20 value 725.496808
## iter 30 value 659.509590
## iter 40 value 656.127374
## iter 50 value 629.472359
## iter 60 value 609.560453
## iter 70 value 594.994162
## iter 80 value 583.555415
## iter 90 value 571.159222
## iter 100 value 546.212619
## final value 546.212619
## stopped after 100 iterations
## # weights: 38
## initial value 915.654962
## iter 10 value 759.556657
## iter 20 value 755.239283
## iter 30 value 671.988845
## iter 40 value 571.233043
## iter 50 value 536.069193
## iter 60 value 531.837741

```

```

## iter 70 value 531.299087
## iter 80 value 525.471326
## iter 90 value 511.811586
## iter 100 value 505.582425
## final value 505.582425
## stopped after 100 iterations
## # weights: 73
## initial value 1458.545007
## iter 10 value 780.333978
## iter 20 value 764.689219
## iter 30 value 763.161005
## iter 40 value 762.185731
## iter 50 value 754.530166
## iter 60 value 742.525158
## iter 70 value 684.449304
## iter 80 value 644.378276
## iter 90 value 554.017151
## iter 100 value 527.277106
## final value 527.277106
## stopped after 100 iterations
## # weights: 108
## initial value 1015.602412
## iter 10 value 780.065809
## iter 20 value 757.777469
## iter 30 value 739.842109
## iter 40 value 707.436182
## iter 50 value 559.543441
## iter 60 value 549.623487
## iter 70 value 532.927617
## iter 80 value 528.850550
## iter 90 value 511.864037
## iter 100 value 506.183689
## final value 506.183689
## stopped after 100 iterations
## # weights: 38
## initial value 962.688436
## iter 10 value 872.109550
## iter 20 value 756.480462
## iter 30 value 686.889063
## iter 40 value 588.207329
## iter 50 value 534.185269
## iter 60 value 527.864121
## iter 70 value 514.637450
## iter 80 value 507.320504
## iter 90 value 507.287505
## iter 100 value 507.276896
## final value 507.276896
## stopped after 100 iterations
## # weights: 73
## initial value 1387.771671
## iter 10 value 869.751170
## iter 20 value 754.474828
## iter 30 value 718.270657
## iter 40 value 673.557703

```

```

## iter 50 value 608.241796
## iter 60 value 549.750786
## iter 70 value 535.842262
## iter 80 value 524.588886
## iter 90 value 503.445556
## iter 100 value 502.592575
## final value 502.592575
## stopped after 100 iterations
## # weights: 108
## initial value 1045.351233
## iter 10 value 773.282740
## iter 20 value 757.328833
## iter 30 value 720.518405
## iter 40 value 603.496260
## iter 50 value 544.468945
## iter 60 value 522.711375
## iter 70 value 517.407345
## iter 80 value 513.036045
## iter 90 value 506.491216
## iter 100 value 505.920161
## final value 505.920161
## stopped after 100 iterations
## # weights: 38
## initial value 1754.169525
## iter 10 value 1030.786047
## iter 20 value 899.389792
## iter 30 value 847.500352
## iter 40 value 814.781957
## iter 50 value 793.107156
## iter 60 value 775.383996
## iter 70 value 767.425771
## iter 80 value 722.461377
## iter 90 value 656.418967
## iter 100 value 644.638478
## final value 644.638478
## stopped after 100 iterations

```

```

end_time_train <- Sys.time()
training_time <- end_time_train - start_time_train
cat("Tiempo de entrenamiento:", training_time, "\n")

```

```

## Tiempo de entrenamiento: 7.566353

```

```

# Hacer predicciones con el conjunto de entrenamiento
predicciones <- predict(modelo_cv, newdata = train)

# Crear la matriz de confusión para el conjunto de entrenamiento
confusion_matrix <- table(Predicho = predicciones, Real = train$PrecioCategoria)
print(confusion_matrix)

```

```

##           Real
## Predicho barata media cara

```

```
##   barata    230   102    1
##   media     66   154   58
##   cara      17    56  253
```

```
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("Accuracy del modelo en entrenamiento:", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy del modelo en entrenamiento: 67.98 %
```

```
# Medir el tiempo de predicción para el conjunto de prueba
start_time_test <- Sys.time()
```

```
# Hacer predicciones con el conjunto de prueba
predicciones_test <- predict(modelo_cv, newdata = test)
```

```
end_time_test <- Sys.time()
testing_time <- end_time_test - start_time_test
cat("Tiempo de predicción:", testing_time, "\n")
```

```
## Tiempo de predicción: 0.003139019
```

```
# Crear la matriz de confusión para el conjunto de prueba
confusion_matrix_test <- table(Predicho = predicciones_test, Real = test$PrecioCategoria)
print(confusion_matrix_test)
```

```
##           Real
## Predicho barata media cara
##   barata    58   24    2
##   media     16   32    9
##   cara       4   21   65
```

```
accuracy_test <- sum(diag(confusion_matrix_test)) / sum(confusion_matrix_test)
cat("Accuracy en el conjunto de prueba:", round(accuracy_test * 100, 2), "%\n")
```

```
## Accuracy en el conjunto de prueba: 67.1 %
```

Podemos ver que nos tardamos 0.018 segundos y 9.19 en la regresion logistica pero en KNN nos tardamos nas eb test siendo 0.04 pero en entrenamiento menos que es 6

Conclusion

La regresion Logistica es bastante buena realizando clasificacion binaria, e incluso en predecir valores. De hecho llegar a superar la regresion lineal. Y con la clasificacion binaria llega a ser muy buena clasificando casi un 100%, sin embargo no presenta una mejora e incluso hasta empeora en terminos no de eficiencia sino de prediccion con otros modelos de clasificacion al utilizar mas de una variable categorica.

Por lo que podemos concluir que la regresion logistica es muy buena solo realizando predicciones o clasificacion binaria. Al enfrentarse con muchas mas variables no mejora nada.